



Bachelor Project Dissertation

Implementation Record

(CNS4949B)

**Title: Transfer Learning Based Network IoT Intrusion Detection
Framework with Audio Feedback**

Name: NURUL FARIZATUL AINA BINTI MOHAMMAD FARIZAL

Matrix Number: 216638

Supervisor: MADAM Y.M. RAJA AZLINA BINTI RAJA MAHMOOD

Semester/ Sesi: 2, 24/25

Date: 13 February 2026

**Bachelor of Computer Science (Computer Network) Faculty of Computer Science
and Information Technology.**

Acknowledgement

Table of Contents

Chapter 1: Introduction

1.1 Introduction.....	11
1.2 Problem Statement.....	12
1.3. Objective	13
1.4. Project Scope	14
1.4.1 IoT Detection using Transfer Learning.....	14
1.4.2 Datasets used and subset creation	14
1.5 Significance of the study	16
1.6 Research Gap	16

Chapter 2: Literature Review **18**

2.1 Growth of IoT and Emerging Security Risks.....	18
2.2 Network Intrusion Detection Systems in Security Operation Center.....	18
2.3 Transfer Learning.....	21
2.4 Comparison between ML, DL and TL.....	22
2.5 IDS detection using Transfer Learning (TL)	23
2.6 Sonification in Network Security for Situational Awareness	28

Chapter 3: Methodology **30**

3.1 Overall System Architecture.....	30
3.2 Dataset used	31
3.2.1 Bot-IoT Dataset.....	31
3.2.2 UNSW-NB15 Dataset.....	35
3.3 Data Preprocessing	39
3.3.1 Data Cleaning.....	39

Label and Text Standardization	39
Handling Missing Values.....	40
Binary Feature Normalization.....	40
Data Type Conversion	40
3.3.2 Feature Selection.....	41
3.3.3 Data Encoding.....	42
3.3.4 Normalization	42
3.3.5 Train-Test Split	43
3.4 Proposed Intrusion Detection Model	45
3.4.1 Base Model Selection	45
3.4.2 Transfer Learning Strategy	47
3.4.3 Model Architecture	48
3.5 Training Configuration	51
3.6 Sonification (Audio Alert) Module	54
3.7 System Integration	55
3.8 Performance Evaluation.....	57
3.8.1 Detection Metrics.....	57
3.9 Development Environment	58
<i>Chapter 4: Implementation.....</i>	62
4.1 Overview	62
4.2 Dataset Preparation	62
4.2.2 Feature Engineering	64
4.2.3 Data Filtering Based on Protocol and Connection State.....	67
4.3 Source Model Implementation (CNN-B)	70
4.3.3 Model Evaluation and Performance.....	74
4.4 Transfer Learning Model (CNN-TL).....	78
4.5 Monitoring Dashboard	83

4.6 Telegram Alert Notification	84
<i>Chapter 5: Results and Discussion</i>	85
5.1 Training Results	85
5.2 Detection Analysis	88
5.3 Performance Analysis	92
5.4 Comparison in evaluation metrics.....	93
<i>Chapter 6: Conclusion and Future Works</i>	96
<i>References</i>	111
<i>Appendix</i>	115

List of Tables

Table No.	Title	Page
Table 1	Comparison of Machine Learning, Deep Learning, and Transfer Learning	19
Table 2	Summary of Transfer Learning IDS Related Works	24
Table 3	Summary of Sonification in Network Security Related Works	28
Table 4	Summary of Features Used from BoT-IoT Dataset	33
Table 5	Summary of Features Used from UNSW-NB15 Dataset	36
Table 6	Data Cleaning Operations Summary	39
Table 7	Selected Features for Intrusion Detection	40
Table 8	Dataset Split Distribution	43
Table 9	Summary of Architecture Components	49
Table 10	Summary of Training Parameters	50
Table 11	Attack-to-Sound Mapping Configuration	54
Table 12	System Hardware Specifications	59
Table 13	Data Filtering Criteria for Protocol and Connection State	68
Table 14	CNN-B Layer Architecture	70
Table 15	CNN-B Training Hyperparameters	72
Table 16	CNN-TL Layer Architecture	79
Table 17	NON-TL Model Detection Samples	92
Table 18	TL Model Detection Samples	94
Table 19	Comparison of NON-TL, TL, and Base Paper	97

List of Figure

Figure No.	Title	Page
Figure 1	Overall System Architecture Framework	30
Figure 2	BoT-IoT Dataset Attack Distribution	33
Figure 3	UNSW-NB15 Dataset Attack Distribution	36
Figure 4	Data Preprocessing Pipeline Flowchart	39
Figure 5	One-Hot Encoding Illustration	42
Figure 6	CNN Architecture for Intrusion Detection	48
Figure 7	Transfer Learning Strategy Visualization	49
Figure 8	Attack-to-Sound Mapping Scheme	54
Figure 9	System Integration Architecture	55
Figure 10	Stage 1 and Stage 2 Implementation Workflow	70
Figure 11	CNN-B Model Architecture Visualization	71
Figure 12	CNN-B Training Loss vs. Epoch	75
Figure 13	CNN-B Training Accuracy vs. Epoch	76
Figure 14	Training and Evaluation of CNN-B to Create CNN-TL	78
Figure 15	CNN-TL Training After UNSW-NB15 Preprocessing	79
Figure 16	Detection Stage Process Flow	81
Figure 17	Confusion Matrix Visualization Code Output	83
Figure 18	Monitoring Dashboard Interface	84
Figure 19	NON-TL Model Confusion Matrix Heatmap	88
Figure 20	TL Model Confusion Matrix Heatmap	90
Figure 21	Per-Class Detection Comparison: NON-TL vs TL	95
Figure 22	Performance Metrics Comparison Chart	98

List of Abbreviation

Abbreviation	Full Form
ACCS	Australian Centre for Cyber Security
Adam	Adaptive Moment Estimation
API	Application Programming Interface
ARP	Address Resolution Protocol
BiLSTM	Bidirectional Long Short-Term Memory
BoT-IoT	Botnet Internet of Things (Dataset)
CBAM	Convolutional Block Attention Module
CEF	Common Event Format
CNN	Convolutional Neural Network
CNN-B	Convolutional Neural Network - Baseline
CNN-TL	Convolutional Neural Network - Transfer Learning
CPU	Central Processing Unit
CSV	Comma-Separated Values
DDoS	Distributed Denial of Service
DL	Deep Learning
DNN	Deep Neural Network
DoS	Denial of Service
FN	False Negative
FP	False Positive
FGSM	Fast Gradient Sign Method
FL	Federated Learning
F1	F1-Score (Harmonic mean of precision and recall)
GAN	Generative Adversarial Network
GDPR	General Data Protection Regulation
GNN	Graph Neural Network
Grad-CAM	Gradient-weighted Class Activation Mapping
GRU	Gated Recurrent Unit
HPC	High Performance Computing
HTTP	Hypertext Transfer Protocol

ICMP	Internet Control Message Protocol
ID	Identifier
IDMEF	Intrusion Detection Message Exchange Format
IDS	Intrusion Detection System
IGMP	Internet Group Management Protocol
IoT	Internet of Things
IP	Internet Protocol
KNN	K-Nearest Neighbors
LIME	Local Interpretable Model-agnostic Explanations
LSTM	Long Short-Term Memory
ML	Machine Learning
MQTT	Message Queuing Telemetry Transport
MTTR	Mean Time to Respond
NIDS	Network Intrusion Detection System
NON-TL	Non-Transfer Learning (Model trained from scratch)
PGD	Projected Gradient Descent
RAM	Random Access Memory
ReLU	Rectified Linear Unit
ResNet	Residual Network
RNN	Recurrent Neural Network
SDG	Sustainable Development Goal
SHAP	SHapley Additive exPlanations
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation, and Response
SOC	Security Operations Center
SSD	Solid State Drive
SVM	Support Vector Machine
TCP	Transmission Control Protocol
TFT	Temporal Fusion Transformer
TL	Transfer Learning
TN	True Negative
TP	True Positive
UDP	User Datagram Protocol

Abstract

Network Intrusion Detection Systems (NIDS) are essential for ensuring the security and reliability of network communications, whether in organizational or personal contexts. With the increasing complexity and frequency of cyber-attacks, timely detection and response are critical to minimizing potential damage.

Most existing NIDS rely primarily on visual alerts and dashboards, which can limit the speed and effectiveness of threat response, especially in time-critical scenarios. Delays in noticing or interpreting alerts may increase the risk of network compromise and reduce overall system resilience in Security Operation Centres (SOC).

In the context of machine learning (ML), the traditional method is a good approach. However adapting with the evolving IoT network dataset faces significant challenges. IoT environments involve heterogeneous device behaviors, lightweight communication protocols, and diverse often malicious traffic patterns, making the data highly dynamic and complex. Due to these factors, conventional ML models that rely heavily on large, task-specific datasets may struggle to generalize effectively in this scenario. Therefore, this project requires an improved method to quickly adapt to the data and achieve a high detection performance. Transfer Learning (TL) is adopted as the core method because it enables knowledge transfer from pre-trained models to new but related tasks. This significantly improves model performance even when the available dataset is scarce. In addition, TL reduces training time, computational cost, and data dependency, making it highly suitable for resource-constrained IoT environments. However, traditional ML could act as a baseline for comparison to the method that would be used. This allows the effectiveness of the TL approach to be clearly evaluated and validated

To relate to the limitations of existing NIDS, this research proposes an IoT-based Network Intrusion Detection framework that leverages transfer learning and incorporates audio mechanisms. By providing real-time audio alerts, the proposed system aims to improve response times, enhance situational awareness, and strengthen the overall effectiveness of intrusion detection in IoT networks in the SOC settings.

Chapter 1: Introduction

1.1 Introduction

The rapid growth of the Internet of Things (IoT) has revolutionized modern technology by connecting billions of devices across various domains such as healthcare, smart homes, transportation, and industry. These interconnected devices communicate and share data through networks, enabling automation and improved efficiency. However, this widespread interconnectivity also introduces new vulnerabilities and security challenges. The absence of standardized security frameworks and the limited computing resources of IoT devices make them easy targets for cyberattacks such as Denial of Service (DoS), data breaches, and unauthorized access.

To counter these threats, **Network Intrusion Detection Systems (NIDS)** have become an essential component in safeguarding IoT infrastructures. NIDSs monitor network traffic, identify malicious activities, and alert administrators to potential attacks. Traditional IDSs are often rule-based or signature-based, which makes them ineffective against new or evolving attack types. In recent years, **Machine Learning (ML)** and **Deep Learning (DL)** approaches have shown great potential in enhancing NIDS performance by enabling systems to learn complex attack patterns and adapt to unseen threats.

One of the promising approaches within deep learning is **transfer learning**, where a pre-trained model is reused and fine-tuned for a specific task. This method allows researchers to leverage existing knowledge from large datasets, significantly reducing training time and improving accuracy which is a key advantage for NIDS applications in IoT environments where resources are limited.

Despite these technological advancements, applications in IoT environments often operate under strict resource constraints, requiring lightweight and efficient security solutions. Despite recent technological progress, a major limitation remains in how cybersecurity monitoring systems deliver alerts to users. Most existing NIDS depend heavily on visual dashboards, graphs, and onscreen notifications. These approaches assume that users are constantly watching a display, which is not always practical in real-world scenarios where attention may be divided or screens

may not be continuously monitored. Additionally, many systems rely on basic sound alerts that lack meaningful context about the type or severity of the detected threat.

To overcome these limitations, this study proposes an NIDS framework that integrates a transfer learning–based detection model with a sonification mechanism for real-time alert delivery. The system translates detected intrusion patterns into distinct audio cues, enabling users to recognize security events without needing to constantly observe a visual interface. By introducing an alternative feedback channel, the system aims to improve situational awareness and responsiveness in cybersecurity monitoring, particularly in dynamic or resource-constrained IoT environments.

1.2 Problem Statement

Although deep learning and transfer learning have significantly improved the accuracy of intrusion detection in IoT networks, most existing systems remain predominantly visual in nature, relying heavily on textual logs, dashboards, and graphical interfaces. In high-demand monitoring environments such as Security Operation Centers (SOCs), analysts are often required to continuously observe multiple screens, which can lead to information overload and delayed response to critical incidents.

Furthermore, limited research has explored the integration of multimodal feedback, such as sound, into Network Intrusion Detection System (NIDS) interfaces. Most IDS frameworks focus primarily on improving detection performance while overlooking how alerts are communicated to users in real time. As a result, important security events may not receive immediate attention if operators are not actively monitoring visual dashboards.

This research therefore identifies the need for an intrusion detection system that not only achieves accurate detection performance through transfer learning but also enhances situational awareness by incorporating alternative sensory feedback mechanisms. By integrating an audio-based alert module, the system aims to provide immediate and intuitive notification of security incidents, complementing traditional visual monitoring methods. Thus, the three main problem statements for this research project are:

- i. Traditional machine learning method often fails to detect new and evolving IoT attacks.
- ii. Transfer learning models used on IoT data need proper evaluation to ensure reliable results.
- iii. Visual dashboards delay response time.

1.3. Objective

The objectives of this research are:

- 1. To implement transfer learning method to detect new attacks more easily.
- 2. To evaluate and compare the proposed transfer learning model using standard IDS metrics.
- 3. To integrate notification alerts mechanism to enhance alertness.

1.4. Project Scope

1.4.1 IoT Detection using Transfer Learning

This project aims to implement a transfer learning method to detect new attacks more efficiently and effectively. The proposed framework is developed by replicating the core detection methodology presented in the study “*Transfer Learning Based Detection Framework in IoT Networks*”, which serves as the foundational model for this research. The replication permits controlled evaluation within the parameters of this study while guaranteeing a validated baseline architecture.

The key objectives of the study involve adapting the proposed model of transfer learning to the available data sets related to the traffic of IoT networks, as well as the extent of the model's ability to successfully generalize the unknown patterns of attacks. The system is software-based mainly, focusing more on the detection of network-related intrusions compared to the use of hardware. This study extends the usability of the original detection framework by the inclusion of the proposed audio-based notification systems, besides recreating the original model. This is because, by converting the results of the process of intrusion detection to sound, it becomes possible to be aware of the related issues in real time.

1.4.2 Datasets used and subset creation

1.4.2.1 Bot-IoT Dataset

The Bot-IoT dataset is the main dataset utilized in this research, acting as the source dataset for model evaluation and training. The Bot-IoT dataset is a large-scale IoT network traffic dataset consisting of approximately 72 million records. The flow-based traffic that was extracted occupies approximately 16.7 GB of space in CSV format. It encompasses a number of attack scenarios that are relevant to IoT contexts, such as keylogging, data exfiltration, operating system and service

scanning, DDoS, and DoS. Moreover, DDoS and DoS attacks can be classified according to their communication protocol, which allows the analysis of attack patterns to be done more deeply. The dataset has been divided into separate 74 different CSV files, making it easier to manage it. Only two subsets of this dataset are chosen for training the model due to the environment's limitations. Even though this approach reduces the number of overall data points, it allows for the representation of attacks and normal traffic patterns needed to test this model of intrusion detection, as discussed earlier.

As such, the scope of the project, which focuses on the construction of the model, adaption of a pre-trained model, and integration of the system as opposed to processing the data or even deployment of the set dataset, aligns with this selection method.

1.4.2.2 UNSW-NB15 Dataset

As part of this research, the use of the UNSW NB15 dataset as the target dataset assisted in the development of the proposed model of intrusion detection. The benchmark dataset for network intrusions, called the UNSW NB15 dataset, was created by simulating a synthetic and real-world network traffic environment. It was designed to accommodate the limitations of the earlier datasets by providing better network behaviors and modern-day attacks.

Within the dataset, there is a total of approximately 2.5 million network flow records. A total of 49 features characterize the dataset. They include flow-based characteristics, as well as other associated content features, time-related characteristics, and other newly created elements which are all associated with intrusion detection. These features encompass general network activities as well as a number of assault types, including worms, shellcodes, reconnaissance, backdoors, analyses, fuzzers, exploits, and DoS attacks. The dataset is associated with generalization for the assessment of machine learning and deep learning-based intrusion detection models.

1.5 Significance of the study

With the suggestion of a more user-aware method of intrusion detection, especially in the context of Security Operations Center (SOC) environments, this study advances the subject of cybersecurity. By allowing security staff to receive and comprehend system alarms via various feedback channels, the system improves cybersecurity monitoring by facilitating the communication of threat information during real-time network surveillance.

Technically, the addition of the transfer learning aids in enhancing the flexibility and effectiveness of the IDS. This helps the system to keep performing satisfactorily in terms of the effectiveness of the detections even when the system is exposed to threats which have previously not been encountered. In addition to this, the inclusion of the audio feedback mechanism aids in the extension of the possibilities associated with the traditional IDS. Within this context, the proposed solution aids in the management of fatigue among the monitors in scenarios where this is a concern. In other instances, the proposed solution aids in awareness being raised more effectively while reducing the need for constant observation.

On a bigger scale, the study promotes a flexible mode of interaction with cybersecurity tools, which is agreeable with the concepts of universal system design. Besides, the study aligns with the UN SDGs as follows:

SDG 9: Industry, Innovation, and Infrastructure - by developing innovative and user-centric cybersecurity monitoring solutions.

In general, the contributions of this study are more flexible, effective, and human-centered cybersecurity monitoring systems, thereby bridging the gap between intrusion detection performance and efficient alarm transmission in SOC operation.

1.6 Research Gap

Intrusion detection systems (IDS) for Internet of Things (IoT) environments have been a point of interest for many researchers, but still, a number of important shortcomings have not been addressed. Many existing intrusion detection system techniques seem to be designed and evaluated using the same data, which does not reflect a true practical situation. The nature and types of network traffic may be different for different systems, and this leads to a problem known as a

domain shift, where one model is not able to generalize well from one data set to the other. This problem of domain shift is usually reported for both deep learning and machine learning techniques.

Further, the majority of the existing IDS research focusing on IoT tackles the issue of improving the accuracy of the detection system while failing to consider the adaptability and reusability of the models. It requires a lot of processing power and data to train models from scratch for a new environment, but it is not always feasible. Methods for assisting the models in the effective transfer of learned knowledge from one network environment to another are needed.

In addition, much of systems' IDS alert systems rely fundamentally on text-based information, logs, and visual-based systems. Information overload is highly possible in situations like Security Operation Centers (SOCs), which can result in a response delay to critical incidents. In cybersecurity monitoring, different alarm systems capable of supporting traditional visual systems have not been sufficiently researched as alternatives.

Therefore, a research gap exists in developing an IDS framework that:

1. Reduces training cost through **knowledge transfer**, and
2. Explores **alternative alert mechanisms** to enhance real-time security monitoring.

Chapter 2: Literature Review

2.1 Growth of IoT and Emerging Security Risks

It is the era of technological advancements, thus the rising concern of technology security, especially IoT networks, is a major concern for every iPhone, Android or any other device with an operating system. For example, in 2017, a casino was hacked through its IoT fish tank thermometer. According to an article written by Wei in 2018, the hackers gained access to the network by taking advantage of a vulnerability in the thermostat. After gaining access to the highrolling gamblers' database, they then pulled it back across the network, out the thermostat, and up to the cloud. Another article from Forbes by Mathews (2017), it was estimated that the hackers had swiped over 10 gigabytes of data and the scary part was that the communication that was monitored by the company's software seemed normal. This is why IoT security is in dire need to be more improved.

2.2 Network Intrusion Detection Systems in Security Operation Center

SOC represents a central department that is responsible for monitoring, detecting, analyzing, and responding to cybersecurity incidents within an organization's network infrastructure. SOCs operate on a 24/7 basis and are heavily dependent on security monitoring tools to maintain situational awareness and protect critical digital assets. This includes, but is not limited to, NIDS, which provides real-time detection of hostile or suspicious network activity.

A network intrusion detection system monitors network traffic and trends that could be a sign of known attacks or potential malicious behavior. The two traditional methods that network intrusion detection systems employ are classified into two bodies of techniques: anomaly detection method and signature detection method. The network traffic patterns are matched up against a database of given attack signatures using the signature detection method. The signature detection method is able to successfully detect attacks that are already known but is unable to detect unknown attacks. Conversely, the anomaly detection method uses normal traffic patterns as input to detect any

anomaly. While anomaly detection is able to detect unknown attacks, it is liable to raise more false alarms.

With the rapid growth of IoT devices and complex network architectures, SOC environments are becoming more data-intensive and dynamic. IoT networks produce large volumes of heterogeneous traffic, often from resource-constrained devices with limited built-in security mechanisms. This increases the attack surface and complicates traffic analysis. Conventional rule-based and manually configured IDS tools are often insufficient to handle such scale and variability, prompting the integration of machine learning and deep learning techniques into SOC monitoring workflows.

Machine learning-based Network Intrusion Detection Systems (NIDS) have emerged as a powerful approach for automatically learning complex patterns from network data, significantly improving the detection of previously unseen attack behaviors compared to traditional signature-based methods [8]. In Security Operations Center (SOC) environments, these intelligent detection models support human analysts by filtering vast amounts of network traffic and highlighting high-risk events, thereby reducing alert fatigue and enabling faster incident response [1, 3, 9]. However, despite these advances, several challenges remain that hinder widespread deployment. These include the requirement for large, labeled datasets for effective training, the need for frequent model retraining to address emerging attack types, and significant computational demands that may not align with real-time operational requirements in production environments [2, 8].

To address these limitations, modern SOC-oriented intrusion detection research explores adaptive approaches such as transfer learning, which enables models to leverage knowledge acquired from previously learned tasks to detect new threats more efficiently with limited training data [2, 5, 7]. Transfer learning approaches enhance scalability and substantially reduce training time by reusing pre-trained model weights, making them particularly suitable for continuously evolving network environments where rapid adaptation to new attack patterns is essential [2, 7]. Recent studies have demonstrated that hybrid architectures combining transfer learning with advanced deep learning techniques, such as Swin Transformers and Long Short-Term Memory networks, can achieve

detection accuracy approaching 99% across multiple benchmark datasets while maintaining computational efficiency [2].

Network Intrusion Detection Systems remain a foundational component of effective SOC operations, serving as the primary technical control for identifying malicious activity within organizational networks [9]. The overall effectiveness of a SOC depends critically on the IDS's ability to provide accurate, timely, and actionable alerts that enable analysts to distinguish genuine threats from benign anomalies [3]. Poor rule quality and high false-positive rates not only increase analyst workload but also degrade trust in the detection system, potentially leading to missed critical alerts [1, 9]. Therefore, contemporary research efforts continue to focus not only on improving detection accuracy through advanced machine learning techniques but also on enhancing how alerts are generated, prioritized, and delivered to SOC analysts to support effective decision-making under time constraints [1, 3, 9]. This includes exploring multimodal alert delivery mechanisms, such as sonification, to complement traditional visual dashboards and improve situational awareness in high-pressure monitoring environments [4, 5, 7].

2.3 Transfer Learning

Traditional machine learning approaches remain limited to manual feature engineering, thereby becoming the major setback for their performance. Although these models tend to have faster training, their average accuracy level remains moderate when dealing with complex tasks such as intrusion detection in the context of IoT. These drawbacks in traditional machine learning approaches are overcome by deep learning approaches, which achieve feature learning from the collected data, resulting in their higher accuracy levels. However, these models take a considerably long time for training.

On the contrary, transfer learning has the best out of both worlds. This method uses pre-trained models such as ResNet and EfficientNet and enables researcher networks to benefit from past features learned, thus significantly decreasing dependence on large data and extended training times.. Transfer learning not only lessens training and development time but also helps in attaining significantly high accuracy in situations with limited data, such as within IoT networks, and hence makes transfer learning one of the most appropriate and effective ways for developing scalable intrusion-detection systems.

2.4 Comparison between ML, DL and TL.

Table 1. Comparison of Machine Learning, Deep Learning and Transfer Learning

Features / Aspect	Traditional Machine Learning	Deep Learning	Transfer Learning
Model examples	SVM, Decision Tree, Random Forest	CNN, RNN, LSTM, DNN	Pre-trained CNNs (e.g., ResNet), BERT, Autoencoders
Feature engineering	Manual (domain knowledge needed)	Automated (learns features from data)	Reuses features from source model
Training time	Fast	Slow (complex models)	Faster (due to reusing pretrained models)
Accuracy	Moderate	High (especially on large datasets)	Very high

Table 1. summarizes the distinctions between Traditional Machine Learning, Deep Learning, and Transfer Learning as reported in existing studies. In the early days of research on intrusion detection, traditional machine learning techniques like Random Forests, Decision Trees, SVM, etc., are commonly adopted. These methods mostly depend on traditional feature engineering methods. Feature engineering typically requires knowledge of the domain for selecting suitable features. Their performance is often limited with complex and high-dimensional network traffic data, although these techniques normally guarantee lower training and processing time.

For instance, convolutional neural networks, recurrent neural networks, long short-term memory networks, and deep neural networks fall within the class of deep learning techniques. These approaches have gained popularity because of the advantage they offer in automatically discovering complex characteristics from raw data. The possibility of accomplishing better detection results is due to the advantage of automatically discovering complex characteristics, which reduces the need for the manual selection of characteristics, especially in large datasets.

Deep learning models have the disadvantage of requiring more computer resources due to the complexity associated with the models.

Transfer learning is a deep learning variation that overcomes these drawbacks. By leveraging pretrained models like ResNet, BERT, and autoencoders, the transfer learning methodology eliminates the requirement for costly training from scratch methods and large amounts of labeled data. Because research shows that the transfer learning approach is highly accurate and requires less training time, it is advantageous in a rapidly expanding field like IoT security, where attack patterns are ever-changing and data may be limited. This comparison analysis clearly shows a research trend from classical machine learning to deep learning and, more recently, transfer learning for the objective of balancing detection and compute efficiency.

2.5 IDS detection using Transfer Learning (TL)

Recent research shows a clear transition from traditional machine learning (ML) approaches toward **deep learning (DL)** and particularly **transfer learning (TL)** techniques in Intrusion Detection Systems (IDS), especially for **IoT network environments**. Conventional ML models such as Decision Trees, Random Forests, KNN, and XGBoost rely heavily on manual feature engineering and often struggle with the highly dynamic and heterogeneous traffic patterns found in IoT networks. As demonstrated by Agrawal et al. (2024), although ML methods can achieve moderate performance, their effectiveness varies widely depending on feature selection and preprocessing. In contrast, DL models such as Convolutional Neural Networks (CNNs) automatically learn hierarchical features from network traffic, resulting in better detection performance. However, TL-based approaches show the most consistent and superior performance because they leverage knowledge learned from large source datasets and adapt it to new intrusion datasets.

Studies such as Rodríguez et al. (2022) and Abdelhamid et al. (2024) demonstrate that pretrained CNN architectures, including VGG, ResNet, MobileNet, and EfficientNet, can be successfully transferred to intrusion detection tasks, achieving very high accuracy levels close to 100% on datasets like UNSW-NB15 and BoT-IoT. These results highlight the strength of TL in improving

feature representation and reducing training cost, especially when labeled security data is limited. Abdelhamid et al. further enhanced TL performance by incorporating attention mechanisms such as CBAM, allowing the model to focus on the most relevant traffic features. This indicates that combining TL with attention-based feature weighting can significantly improve detection capability in complex IoT traffic environments.

Beyond standalone TL models, some studies explore hybrid frameworks that integrate IDS with emerging technologies. For example, Alkadi et al. (2020) proposed a deep learning and blockchain-enabled collaborative IDS to secure IoT and cloud networks, aiming to enhance trust and decentralization. Similarly, Fan et al. (2020) introduced a federated transfer learning framework to protect distributed IoT devices while preserving data privacy. Ullah and Mahmoud (2021) experimented with multi-dimensional CNN models (1D, 2D, and 3D) combined with TL to capture different traffic representations. Although these hybrid approaches address scalability and privacy concerns, many do not clearly report performance metrics or are computationally complex, which may limit their practical deployment in real-world environments such as Security Operation Centers (SOC).

Most of the reviewed studies utilize modern intrusion datasets, particularly UNSW-NB15 and BoT-IoT, which include diverse attack types such as DoS, DDoS, scanning, and botnet activities. These datasets better represent contemporary network threats compared to older datasets like NSLKDD. The consistent use of these datasets across the literature supports their suitability for evaluating TL-based IDS models and ensures that research results remain comparable.

Despite the high accuracies reported, several research gaps remain. Many studies emphasize model performance but provide limited discussion on how IDS outputs integrate into SOC environments for real-time monitoring and incident response. Additionally, large pretrained models such as VGG and ResNet may not be efficient for real-time deployment due to computational demands. Crossdomain adaptability, one of the key motivations for TL, is also not thoroughly evaluated in many works. Furthermore, deep learning models often lack interpretability, which can be a challenge for security analysts who need to understand alerts.

Overall, the literature indicates that transfer learning significantly enhances IDS performance in IoT networks by improving generalization, reducing data requirements, and enabling deep feature

extraction. However, practical considerations such as efficiency, SOC integration, and explainability remain underexplored. These limitations justify the need for further research into TL-based IDS frameworks that are not only accurate but also suitable for operational security environments.

Table 2. Summary of related works

Authors	Title	Approach	Algorithm	Dataset	Accuracy (%)
1.Rodríguez et al. (2022)	TransferLearning-Based Intrusion Detection Framework in IoT Networks	TL	Pretrained CNN	Custom + UNSW-NB15, BoT-IoT	99.85-100
2.Agrawal et al. (2024)	Intrusion Detection for Cyber Security: A Comparative Study of Machine Learning, Deep Learning and Transfer Learning Methods	ML/DL/TL	DT, RF, KNN, XGB, CNN	UNSW-NB15, BoT-IoT	45.51–99.74

3. Abdelhamid et al. (2024)	Attention-Driven Transfer Learning Model for Improved IoT Intrusion Detection	TL	VGG16, RestNet50, MobileNetV1, EfficientNetB0 + CBAM	UNSW_NB15, BoT-IoT	99.93
4. Alkadi et al. (2020)	A Deep Blockchain Framework-Enabled Collaborative Intrusion Detection for Protecting IoT and Cloud Networks	DL + Blockchain	BiLSTM + Smart Contracts	UNSW-NB15, BoT-IoT	Not clearly stated (claimed better than baselines)
5. Ullah and Mahmoud (2021)	Design and Development of a Deep Learning Based Model for Anomaly Detection in IoT Networks	DL + TL	CNN (1D, 2D, 3D + TL)	BoT-IoT, IoT-23, MQTTIoT-IDS2020, IoT Network Intrusion Dataset	Not explicitly stated (reported high accuracy)

3.Abdelhamid et al. (2024)	Attention- Driven Transfer Learning Model for Improved IoT Intrusion Detection	TL	VGG16,RestNet50,MobileNetV1, EfficientNetB0 + CBAM	UNSW_NB15, BoT-IoT	99.93
-------------------------------	---	----	---	-----------------------	-------

2.6 Sonification in Network Security for Situational Awareness

Overall, the research considered reveals that security operators can recognize irregularities without depending solely on the usage of visual dashboards by interpreting network data through audio signals. The SoNSTAR system, which uses real-time TCP traffic and transforms them into sound elements, was initially proposed by the research work of Debashi and Vickers (2023). In their method, it was revealed that the level of situational awareness can be improved without increasing the workload of users; instead, results revealed that the identification of recurring traffic flows can be successfully achieved through the utilization of unique sound patterns with a very high accuracy rate of 99.92%.

Likewise, Axon et al. (2019) focused on the degree to which humans could recognize network assaults through the utilization of auditory representations. Sound was connected to certain network dimensions in the experiment. The findings had an F-score of 0.93, indicating the ability of a participant to recognize attacks simply through listening. The findings promoted the use of multimodal monitoring, focusing on the cognitive ability of humans to recognize cybersecurity measures through auditory means.

Moreover, Axon et al. (2018) were particularly interested in the opinions of professionals working within Security Operations Centers (SOCs). Sonification allows network administrators to be aware of the potential dangers while multitasking, according to the results of the surveys and interviews carried out by the researchers. As the participants explained, they were able to keep an eye on the state of the network without focusing on the results of visualization. In other words, the efficiency of operation and a lack of overreliance on the visual can be achieved through sonification.

In conclusion, it has been established through research that sonification promotes increased situational awareness, real-time identification of any given threats, and the ability to monitor and detect threats while reducing cognitive strains. However, a significant portion of the research tends to focus more widely upon human interpretation of sonification, rather than fully integrating it with a broader scope of automated machine learning algorithms used in IDS systems.

Table 3. Summary of related works for Sonification in Network Security for Situational Awareness

Authors	Title	Approach	Goal	Discovery
1. Debashi and Vickers (2023)	SoNSTAR: sonification of Network Traffic for Cyber Situational Awareness	Real-Time TCP mapping	Raises situational awareness while imposing less workload.	99.92% accuracy rate by hearing the "repetitive flows" sound.
2. Axon et al. (2019)	Hearing attacks in network data: An effectiveness study	Sound parameters are mapped to different dataset dimensions	Network attacks could be detected accurately by listening to the sonification	The F-score (0.93) shows that participants were able to detect attacks accurately
3. Axon et al. (2018)	Sonification in Security Operations Centres: What do Security Practitioners Think?	Sonification (Survey + Interview)	Allows network administrators to stay alert to threats while multitasking.	Participants stated that they were able to multitask in their role monitoring.

Chapter 3: Methodology

3.1 Overall System Architecture

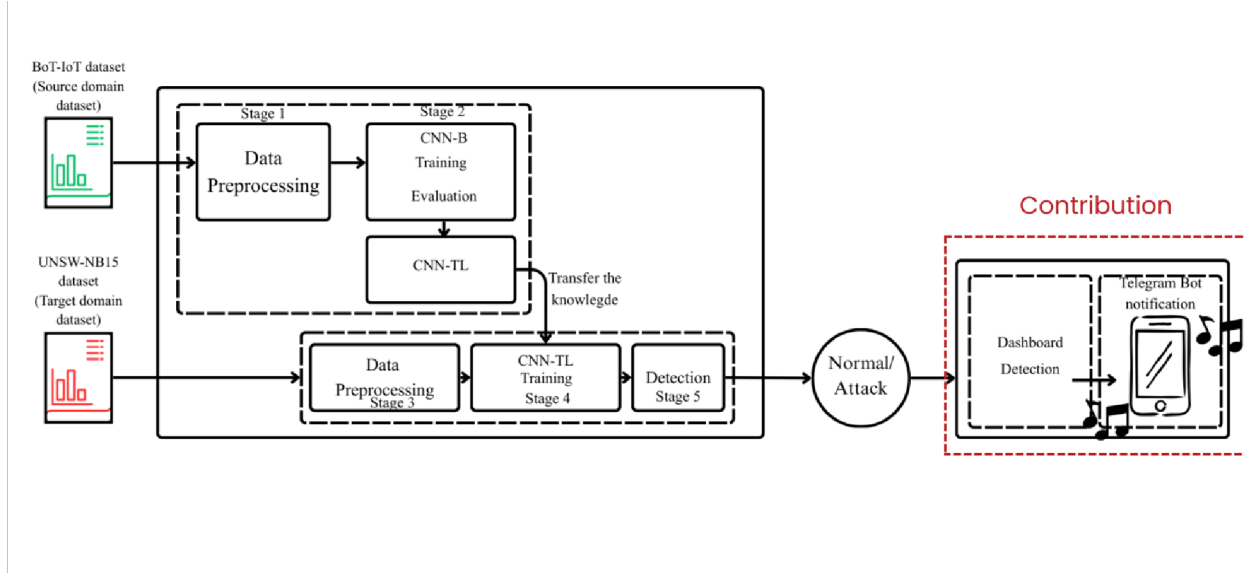


Figure 1. The TL framework from the base paper alongside the project's contribution.

Figure 1. illustrates the proposed transfer learning (TL) based Intrusion Detection System framework adapted from the base study and enhanced with the project's additional alerting contribution. The suggested transfer learning (TL)-based Intrusion Detection System framework, modified from the basis study and improved with the project's extra alerting contribution, is shown in Figure 3. The framework functions in two primary domains: the UNSW-NB15 dataset is used in the target domain, and the BoT-IoT dataset is used in the source domain.

In **Stage 1**, raw network traffic data from the BoT-IoT dataset undergoes data preprocessing, which includes feature selection, normalization, and cleaning to ensure data consistency. In **Stage 2**, a baseline Convolutional Neural Network model (CNN-B) is trained and evaluated using the processed source dataset. This stage enables the model to learn general traffic behavior and attackrelated feature representations. The learned weights from this stage form the knowledge base for transfer learning.

The extracted knowledge is then transferred to the target domain model (CNN-TL). In **Stage 3**, the UNSW-NB15 dataset is preprocessed using the same transformation procedures to maintain feature compatibility. In **Stage 4**, the CNN-TL model is fine-tuned using the target dataset, where

earlier layers retain transferred knowledge while later layers adapt to the new data distribution. This process enhances the model's capability to detect unfamiliar or evolving attacks with reduced training time.

In **Stage 5**, the trained model performs intrusion detection by classifying incoming traffic as either **normal** or **attack**. The detection results are then forwarded to the monitoring component. As an extension to the base framework, the system integrates a dashboard-based monitoring interface and a Telegram bot notification module, which provides real-time alerts. This addition improves operational usability within Security Operation Center (SOC) environments by enabling rapid awareness and response to detected threats.

Overall, the framework combines transfer learning efficiency with real-time alert mechanisms, resulting in an IDS that is both technically effective and operationally practical.

3.2 Dataset used

3.2.1 Bot-IoT Dataset

The BoT-IoT dataset is used as the primary source dataset for this study. Only a sample of the data was selected for model creation as the entire dataset was unavailable for model creation due to resource constraints. To ensure that the model receives enough data variations for learning while keeping the process efficient, the dataset was sampled by taking 1,000,000 records out of the initial data. The 34 features of the initial data cover various aspects of the network traffic, ranging from flow-based features to packet data along with information on IoT devices. The features prove suitable for developing an IDS as crucial data is available to distinguish the data as malicious or valid.

Table 4. Summary of features used from BoT-IoT dataset.

Feature Group	Feature Name	Description	Data Type	Relevance to IDS
Flow ID	pkSeqID	Packet sequence identifier	Integer	Uniquely identifies traffic records
	seq	Flow sequence number	Integer	Tracks packet order in flows
	saddr	Source IP address	Categorical	Identifies traffic origin
	daddr	Destination IP address	Categorical	Identifies traffic target
	sport	Source port	Categorical	Helps detect servicebased attacks
	dport	Destination port	Categorical	Identifies targeted services
Time-Based	stime	Start time of flow	Float	Used to analyze traffic timing behavior
	ltime	Last time of flow	Float	Helps compute flow duration
	dur	Duration of flow	Float	Abnormal durations may indicate attacks
Protocol / State	proto	Network protocol (TCP/UDP/ICMP)	Categorical	Attack behavior differs by protocol
	flgs	TCP flags	Categorical	Useful for detecting scanning and DoS
	state	Connection state	Categorical	Shows session behavior patterns
Traffic Volume	pkts	Total packets in flow	Integer	High packets = possible flooding
	bytes	Total bytes transferred	Integer	Detects abnormal data transfers

	spkts	Source-to-destination packets	Integer	Directional traffic analysis
	dpkts	Destination-to-source packets	Integer	Response behavior monitoring
	sbytes	Bytes sent by source	Integer	Detects data exfiltration
	dbytes	Bytes sent by destination	Integer	Detects unusual responses
Statistical	mean	Mean packet size/time	Float	Captures average traffic behavior
	stddev	Standard deviation	Float	Measures traffic variability
	sum	Sum of values in flow	Float	Overall traffic magnitude
	min	Minimum value	Float	Detects abnormal small bursts
	max	Maximum value	Float	Detects traffic spikes
Rate-Based	rate	Overall transmission rate	Float	High rate = DoS/DDoS indicator
	srate	Source transmission rate	Float	Detects attacker-side flooding
	drate	Destination transmission rate	Float	Detects response anomalies
Labels	attack	Binary attack label	Integer	Target variable for ML model
	category	Attack type category	Categorical	Multi-class classification
	subcategory	Detailed attack type	Categorical	Fine-grained attack analysis

Table 3. presents a comprehensive summary of the 34 features selected from the BoT-IoT dataset for model training. The features are organized into six functional groups based on their role in intrusion detection. The Flow ID group uniquely identifies traffic records and tracks packet sequences, essential for reconstructing communication sessions. Time-based features capture temporal traffic patterns, where abnormal durations may indicate attacks such as slow-rate DoS. Protocol and state information helps differentiate attack behaviors across different protocols (TCP, UDP, ICMP). Traffic volume features detect anomalies in packet counts and byte transfers, crucial for identifying flooding attacks. Statistical features capture traffic variability through mean, standard deviation, min, and max values helping detect subtle anomalies. Rate-based features identify high-rate attacks like DDoS. Finally, label features provide ground truth for supervised learning, supporting both binary and multi-class classification. This comprehensive feature set ensures the model can capture diverse attack signatures while maintaining sensitivity to normal traffic variations.

3.2.2 UNSW-NB15 Dataset

The **UNSW-NB15 dataset** is used as a benchmark dataset to evaluate the robustness and generalization capability of the proposed intrusion detection model. This dataset was developed by the Australian Centre for Cyber Security (ACCS) and contains modern synthetic network traffic that reflects both normal activities and a wide range of contemporary cyberattacks. The dataset consists of **49 features**, including flow-based attributes, content features, time-related features, and additional statistical network characteristics that describe communication behavior in detail.

UNSW-NB15 includes multiple attack types, such as Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic attacks, Reconnaissance, Shellcode, and Worms, covering diverse threat behaviors across different layers of the network. These attacks differ in structure and execution patterns from IoT-focused attacks, making the dataset valuable for testing whether the model can detect threats beyond a single environment.

The purpose of using UNSW-NB15 is to serve as a **comparison and generalization dataset**, allowing the performance of the model trained on IoT traffic (BoT-IoT) to be evaluated on a more general network intrusion dataset. This helps assess the model's ability to adapt to different traffic characteristics and improves the credibility of the proposed IDS by demonstrating effectiveness across multiple datasets.

Table 5. Summary of features used from UNSW-NB15 dataset.

Feature Group	Feature Names	Description	Data Type	Relevance to IDS
Flow Identification	srcip, sport, dstip, dsport	Source/destination IP addresses and ports	Categorical	Identifies communication endpoints and targeted services
Protocol & State	proto, state, service	Network protocol, connection state, and service type	Categorical	Attack behavior differs across protocols and services

Time-Based	dur, stime, ltime	Flow duration and timing information	Numerical	Abnormal session timing may indicate attacks
Traffic Volume	sbytes, dbytes, spkts, dpkts	Bytes and packets exchanged	Numerical	High volume or imbalance indicates flooding or exfiltration
TTL & Loss	sttl, dttl, sloss, dloss	Time-to-live values and packet loss counts	Numerical	Irregular values may signal malicious traffic
Load Features	sload, dload	Source and destination data transfer rates	Numerical	Detects DoS/DDoS behavior
Window & TCP Features	swin, dwin, stcpb, dtcpb	TCP window sizes and sequence numbers	Numerical	Used to detect TCP-based attacks
Packet Size Stats	smeansz, dmeansz	Mean packet sizes	Numerical	Abnormal sizes indicate unusual traffic patterns
HTTP / Content Features	trans_depth, res_bdy_len, ct_flw_http_mthd	HTTP transaction depth and response body length	Numerical	Useful for detecting applicationlayer attacks
Jitter & Latency	sjit, djit, sintpkt, dintpkt	Packet jitter and interpacket timing	Numerical	Irregular timing behavior

				signals anomalies
TCP Timing	tcprtt, synack, ackdat	Round-trip time and handshake timing	Numerical	Helps detect SYN floods and TCP anomalies
Behavioral Flags	is_sm_ips_ports, is_ftp_login	Indicators of suspicious patterns or FTP login attempts	Numerical (Binary)	Detects abnormal connection behavior
Connection Count Features	ct_state_ttl, ct_srv_src, ct_srv_dst, ct_dst_ltm, ct_src_ltm, ct_src_dport_ltm, ct_dst_sport_ltm, ct_dst_src_ltm	Counts of connections within time windows	Numerical	Identifies scanning and repeated attack attempts
FTP Features	ct_ftp_cmd	FTP command tracking	Categorical	Detects FTPbased attacks
Labels	attack_cat, label	Attack category and binary class label	Categorical / Numerical	Target variables for classification
Partially Missing Features	ct_flw_http_mthd, is_ftp_login	Features with missing values	Numerical	Require preprocessing before training

Table 4. details the 49 features from the UNSW-NB15 dataset, organized into 14 functional categories that reflect modern network traffic complexity. The table serves two purposes: first, it demonstrates the dataset's comprehensiveness in covering contemporary attack vectors; second, it highlights the feature compatibility challenges addressed by transfer learning. Key feature groups include flow identifiers for endpoint tracking, protocol-state combinations for behavioral analysis, time-based features for temporal anomaly detection, and volume statistics for identifying bandwidth attacks. Notably, the table includes partially missing features (`ct_flw_http_mthd`, `is_ftp_login`) requiring preprocessing attention. The diversity of features from TCP-specific attributes (window sizes, sequence numbers) to application-layer content (HTTP depth, FTP commands) makes UNSW-NB15 an ideal target dataset for testing the model's generalization capability after transfer from BoT-IoT.

3.3 Data Preprocessing

Due to the various types of traffic patterns and devices used, data collected from networks tends to be inconsistent and varied. Effective modeling also entails ensuring the data collected and used for model creation is consistent and clean. In a bid to enhance the reliability and quality of the data for effective intrusion detection, preprocessing was employed for data cleaning, conversion, and normalization of various data attributes, including host information, network packets, and FTP commands. In Stage 1, raw data collected from the BoT-IoT dataset under network traffic is effectively preprocessed for feature selection, normalization, and cleaning for data consistency. From the preprocessed source data, a baseline model, CNN-B, is constructed and tested for effectiveness in Stage 2. The model has the potential for general learning and feature representation associated with attacks under Stage 2. The knowledge base for transfer learning was formed from the weights obtained during Stage 2.

3.3.1 Data Cleaning

Data preprocessing is a crucial step in machine learning-based intrusion detection systems, as raw network traffic datasets often contain inconsistencies, missing values, redundant labels, and formatting irregularities. Without proper preprocessing, these issues may negatively affect model performance and lead to inaccurate classification results.

In this study, preprocessing was performed to ensure data consistency, reliability, and suitability for deep learning training.

Label and Text Standardization

Categorical features and labels were standardized to maintain uniformity across the dataset. This included:

- Removing leading and trailing whitespace
- Eliminating unnecessary internal spaces
- Converting all text values to lowercase

For example, inconsistent attack labels such as “*backdoors*” were standardized to “*backdoor*” to prevent duplication of categories caused by minor naming variations. This step ensures that each attack type is represented consistently during classification.

Similarly, placeholder symbols such as “-” in categorical fields were replaced with standardized values (e.g., “None”) to improve data clarity and avoid ambiguity.

Handling Missing Values

Several features contained missing (null) values. Since deep learning models require complete numerical input, missing values were handled carefully:

- Numerical features were filled with **0** where appropriate.
- Binary indicators were converted into consistent integer representations (0 and 1).

Replacing missing values prevents runtime errors and ensures that the dataset remains mathematically valid for training.

Binary Feature Normalization

Certain binary features contained values greater than 1 due to dataset inconsistencies. These were normalized to a strict binary format:

- Values greater than 1 were converted to 1
- This ensures logical consistency (e.g., login status = 0 or 1 only)

Maintaining a strict binary representation improves interpretability and prevents unintended bias in model training.

Data Type Conversion

To ensure compatibility with machine learning frameworks, relevant features were converted into appropriate data types:

- Numerical features were converted to integer format

- Categorical inconsistencies were resolved

This step guarantees that all features are processed correctly by the CNN model during training.

3.3.2 Feature Selection

Feature selection is an essential step in building an effective intrusion detection system, as not all attributes in a raw network dataset contribute meaningfully to attack detection. Irrelevant or redundant features may increase computational complexity, introduce noise, and negatively affect model performance.

In this study, the dataset was filtered to retain only the most relevant flow-based features associated with network communication behavior. The selected attributes include:

Table 6. Selected Features for Intrusion Detection

Feature Category	Feature Name	Description / Purpose
Protocol Information	proto	Indicates the communication protocol used in the network flow (e.g., TCP, UDP).
Source & Destination Identifiers	srcip	Source IP address of the network flow.
	sport	Source port number.
	dstip	Destination IP address of the network flow.
	dsport	Destination port number.
Traffic Statistics	spkts	Number of packets sent from source to destination.
	dpkts	Number of packets sent from destination to source.
	sbytes	Number of bytes transmitted from source.
	dbytes	Number of bytes transmitted from destination.
Connection State Information	state	Represents the connection state (e.g., established, closed).
Temporal Features	stime	Start time of the network flow.
	ltime	End time of the network flow.
	dur	Duration of the network connection.
Classification Labels	label	Binary label indicating normal (0) or attack (1).

	<code>attack_cat</code>	Specific category of attack type.
--	-------------------------	-----------------------------------

These features capture essential characteristics of network flows, including packet volume, byte transfer patterns, communication duration, and protocol behavior, which are critical indicators for distinguishing between normal and malicious traffic.

3.3.3 Data Encoding

Data encoding was performed to convert categorical features into numerical representations suitable for deep learning models. Since Convolutional Neural Networks (CNNs) require numerical input, textual categories such as protocol type (`proto`) and service (`service`) cannot be processed directly. Therefore, one-hot encoding was applied using the `pd.get_dummies()` function.

One-hot encoding transforms each categorical value into a separate binary column, where a value of 1 indicates the presence of a specific category and 0 indicates its absence. This approach ensures that no artificial ordinal relationship is introduced between categories, which could otherwise mislead the learning process. For example, protocol types such as TCP, UDP, and ICMP are treated as independent categories rather than ranked values.

Although label encoding is another possible technique, it assigns integer values to categories, which may incorrectly imply a numerical hierarchy. Therefore, one-hot encoding was selected to preserve categorical independence and improve model reliability.

By converting categorical attributes into structured numerical form, the dataset becomes fully compatible with the CNN architecture, ensuring effective feature learning during model training.

3.3.4 Normalization

After feature selection and encoding, data normalization was performed to ensure that all numerical features were scaled to a consistent range. Network traffic attributes such as packet counts, byte sizes, and duration values may vary significantly in magnitude. Without

normalization, features with larger numerical ranges could dominate the learning process and negatively affect model convergence.

In this study, feature scaling was applied using normalization techniques to standardize the input values before feeding them into the CNN model. This process transforms feature values into a comparable scale, typically within a defined range such as 0 to 1.

Normalization improves model performance by:

- Preventing bias toward features with larger numerical values
- Accelerating convergence during training
- Enhancing numerical stability
- Reducing the risk of gradient explosion or vanishing gradients

By ensuring that all input features share a consistent scale, the CNN model can learn meaningful traffic patterns more effectively and achieve improved classification accuracy.

3.3.5 Train-Test Split

To ensure reliable and unbiased model training and evaluation, each dataset used in this study was divided into three subsets: training, validation, and testing sets. The initial split was performed using the `train_test_split` function from Scikit-learn, where 75% of the data was allocated for training and 25% for testing. Shuffling was enabled during the split to ensure that samples were randomly distributed, thereby reducing potential bias caused by data ordering.

The training set is used to train the CNN model and update its weights through backpropagation. During this phase, the model learns to distinguish between normal and malicious network traffic by identifying patterns within the input features. To further enhance model reliability, a portion of the training data was allocated as a validation set during training. The validation set is not used to update model weights but instead serves as a performance monitoring mechanism across training epochs. It plays a critical role in hyperparameter tuning, detecting overfitting, and selecting the best-performing model configuration.

The testing set is reserved exclusively for final evaluation after the training process is complete. This ensures that performance metrics reflect the model's ability to generalize to unseen data, simulating real-world deployment conditions. By maintaining clear separation between training, validation, and testing subsets, the study ensures methodological rigor and prevents data leakage, thereby improving the credibility of the experimental results.

3.4 Proposed Intrusion Detection Model

3.4.1 Base Model Selection

Selection of a proper base model is one of the key steps for creating a successful intrusion detection system. In this work, a CNN architecture was adopted as the base model to perform the classification task of network traffic.

The CNNs, deep learning models initially developed to process spatial data, had later successfully been adapted for the structured data of cybersecurity. In network intrusion detection, traffic metrics such as packet statistics, protocol behaviour, and flow characteristics can demonstrate underlying patterns and inter-correlations. Since the convolutional layer in CNNs automatically learns local feature patterns and forms the hierarchical representation with little manual feature engineering, CNNs are well-suited to capture these associations.

Considering various aspects, it can be stated that CNN performs better in traffic classification. For instance, considering various factors, it can be stated that intrusion detection is different from normal traffic due to statistical distribution, making it a pattern recognition task. It is worth noting that CNN can recognize various types of patterns with better accuracy when compared to other algorithms, as it applies convolutional filters. Furthermore, CNN can be effective in terms of computing power and avoiding overfitting with reduced dimensionality using pooling.

By preserving the spatial correlation between features while organizing the traffic data into matrices for CNNs, the algorithm surpasses the performances of traditional deep neural networks (DNNs) for feature extraction. In addition, CNNs present advantages over other fully connected deep networks regarding the low number of parameters used, thus enhancing the overall training process. CNNs are computationally efficient compared to sequence-based traffic features. They are appropriate for use when traffic features are not sequence dependent.

Recent IDS-related work also prominently employed CNNs, and they were found to possess superior detection capabilities for various benchmark datasets. Due to their architectural features, it is possible to fine-tune and apply these features for newly created incursion data sets.

3.4.2 Transfer Learning Strategy

With the intention of strengthening the efficiency of intrusion detection and reducing the intricacy of training when adjusting to novel network traffic datasets, the current research relies on the use of the transfer learning (TL) approach. So, with the intention of increasing the model's capability to adapt to novel attack types, the method relies on the utilization of prior knowledge obtained from an initial source dataset.

Pre-trained Model on Source Dataset

Accordingly, the BoT-IoT dataset, which contains rich IoT network traffic and various attack types including DDoS, DoS, scanning, and data exfiltration, is exploited to train the first CNN model, also referred to as the source model. In this phase, the CNN model captures low-level features including statistical anomalies, traffic distribution, and protocol features in addition to basic traffic behavior. A sound foundation is provided by the trained weights using the convolutional layers.

Fine-Tuning on Target IoT Dataset

The pre-learnt model is then presented to the target data set, which is UNSW-NB15, following the training process on the source data set. The target data set, representing a different domain, consists of various types of traffic data along with attack behaviors. The retraining of a few selected layers of the pre-learnt CNN using the data from the target set is known as fine-tuning. Since the capabilities of the data set from the source domain have already been learned, fine-tuning helps to adjust the higher-level features of the CNN in line with the properties of the data set from the target domain. This will minimize the requirement for models to be learned from scratch, and accuracy will be improved.

Frozen vs. Trainable Layers

The CNN layers must be divided into trainable and frozen parts in order to apply the transfer learning efficiently:

Freezing layers: While training on the target dataset, initial convolutional layers are not moved. These layers extract generic feature representations which can be utilized effectively across different network contexts, for example, fundamental traffic behavioral patterns. In this way, freezing these layers will prevent the loss of previously learnt information and also save computational costs.

Trainable layers: We set all the fully connected and subsequent convolutional layers to be trainable. These layers learn the higher-order and dataset-specific patterns. Updating these layers may allow the model to adapt to new attack characteristics or traffic distributions of the target dataset.

3.4.3 Model Architecture

The proposed intrusion detection model is implemented using a **Convolutional Neural Network (CNN)** architecture designed to classify network traffic into normal or attack categories. The model consists of an input layer, multiple convolutional and hidden layers for feature extraction, and an output layer for classification.

Input Layer

The input layer receives preprocessed network traffic features extracted from the dataset. Each traffic record is represented as a numerical feature vector after encoding and normalization. To enable convolution operations, the feature vector is reshaped into a two-dimensional matrix format. This allows the CNN to learn spatial relationships among features.

Convolutional and Hidden Layers

The core of the model consists of several **convolutional layers** followed by pooling and dense layers:

- **Convolutional Layers:**

These layers apply convolutional filters to the input data to extract local patterns and feature correlations. Each convolutional layer learns increasingly complex representations of network traffic behavior.

- **Activation Function (ReLU):**

After each convolution, the **Rectified Linear Unit (ReLU)** activation function is applied to introduce non-linearity. ReLU helps the model learn complex patterns efficiently and mitigates the vanishing gradient problem.

- **Pooling Layers:**

Max-pooling layers are used after convolutional layers to reduce the dimensionality of feature maps. This reduces computational complexity and helps prevent overfitting by retaining only the most important features.

- **Fully Connected (Dense) Layers:** The flattened feature maps are passed to one or more dense layers. These layers perform high-level reasoning and combine extracted features to make classification decisions.

Output Layer

The final layer is a dense output layer responsible for classification. A **Softmax activation function** is used to produce probability scores for each traffic class (e.g., normal traffic and different attack categories). The class with the highest probability is selected as the prediction.

Table 7. Summary of Architecture Components

Layer Type	Function
Input Layer	Accepts preprocessed traffic features
Convolutional Layers	Extract local feature patterns
ReLU Activation	Introduces non-linearity
Pooling Layers	Reduce dimensionality
Dense Layers	Perform high-level classification
Output Layer (Softmax)	Produces class probabilities

3.5 Training Configuration

The CNN models were trained using carefully selected hyperparameters to ensure stable convergence and optimal classification performance. The training configurations for both the baseline CNN-B model and the transfer learning CNN-TL model are summarized in Table 7, with the rationale for each parameter discussed in detail below.

Table 8. Summary of Training parameters

Model	Epochs	Batch size	Optimizer	Learning Rate
CNN-B	25	208	Adam	5×10^{-4}
CNN-TL	15	2048	Adam	5×10^{-5}

Loss Function

The categorical cross-entropy loss function was employed for both models to measure the discrepancy between predicted class probabilities and true labels. Since intrusion detection is fundamentally a classification task—distinguishing between normal traffic and various attack types—cross-entropy is particularly suitable as it heavily penalizes incorrect predictions while encouraging the model to assign higher probabilities to the correct classes. For the binary classification variant (normal versus attack), binary cross-entropy would be appropriate, but given the multi-class nature of the intrusion detection task in this study, categorical cross-entropy was applied to handle the multiple attack categories present in both datasets.

Optimizer

The Adam (Adaptive Moment Estimation) optimizer was selected to update model weights during training for both CNN-B and CNN-TL configurations. Adam combines the advantages of two popular optimization algorithms: Momentum, which accelerates convergence by smoothing gradient updates, and RMSProp, which adapts learning rates based on the magnitude of recent gradients. This combination enables faster convergence while maintaining adaptive learning rates for each parameter individually. Adam is widely adopted in deep learning research due to its computational efficiency, robustness to noisy gradients, and minimal hyperparameter tuning requirements—characteristics particularly valuable when working with large-scale network traffic datasets that may contain inherent variability and noise.

Learning Rate

The learning rate, which determines the magnitude of weight adjustments during each optimization step, was configured differently for each model to account for their distinct training objectives. For the CNN-B model trained from scratch on the source domain, a learning rate of 5×10^{-4} was selected. This relatively moderate rate provides sufficient step size for efficient initial learning while maintaining stability to prevent overshooting the loss function minimum. For the CNN-TL model during fine-tuning, a lower learning rate of 5×10^{-5} was employed. This reduced rate is essential when adapting pretrained weights to prevent catastrophic forgetting—the phenomenon where rapid weight updates destroy useful features learned from the source domain. The smaller updates allow the model to gently adapt to the target domain's characteristics while preserving the general traffic representations transferred from CNN-B.

Epochs

The number of training epochs was determined empirically by monitoring validation performance to ensure convergence without overfitting. The CNN-B model was trained for 25 epochs on the BoT-IoT source dataset, which provided sufficient iterations for the randomly initialized network to learn meaningful traffic patterns and attack signatures. The training curves presented in Chapter 5 demonstrate that this duration allowed the model to achieve stable convergence with minimal generalization gap between training and validation losses. For the CNN-TL model, only 15 epochs were required during fine-tuning on the UNSW-NB15 target dataset. The reduced epoch count reflects the primary advantage of transfer learning: by starting from pretrained weights rather than random initialization, the model achieves convergence significantly faster, requiring approximately 40% fewer training iterations while achieving superior performance.

Batch Size

Batch size, which determines the number of samples processed before each weight update, was configured differently for each model based on dataset characteristics and computational considerations. The CNN-B model utilized a batch size of 208 samples, balancing computational efficiency with gradient stability during initial training from scratch. This moderate batch size provides sufficiently accurate gradient estimates while maintaining some stochasticity to help

escape local minima. For the CNN-TL model, a substantially larger batch size of 2048 samples was employed during fine-tuning on the UNSW-NB15 dataset. The larger batch size offers several advantages in the transfer learning context: it provides more stable gradient estimates when making small adjustments to pretrained weights, improves computational throughput by better utilizing parallel processing capabilities, and helps average out noise in the target dataset. The increased batch size is feasible because fine-tuning requires fewer epochs, and the computational efficiency gained helps offset the memory requirements of processing larger batches simultaneously.

The combination of these hyperparameters—lower learning rate, fewer epochs, and larger batch size for CNN-TL compared to CNN-B—reflects the fundamentally different training paradigms. Training from scratch requires more aggressive learning over more iterations to build representations from random initialization, while fine-tuning demands gentle, stable updates over fewer iterations to adapt existing knowledge to new domains. The success of this configuration is validated by the experimental results presented in Chapter 5, where CNN-TL achieves superior performance with significantly reduced training time.

3.6 Sonification (Audio Alert) Module

3.6.1 Design Principle

The alert mechanism in this system is based on the principle of **sonification**, where system information is represented through sound characteristics. Instead of relying solely on visual notifications, the proposed design translates intrusion detection outputs into distinct auditory cues. This approach allows security events to be perceived immediately through sound, supporting faster awareness in monitoring environments.

In this design, different categories of detected network attacks are mapped to specific sound attributes. Each type of intrusion produces a unique auditory pattern, enabling differentiation between normal traffic and various attack classes. The mapping strategy is based on altering fundamental sound properties such as:

- **Pitch** – Higher pitch tones may represent more severe or critical attacks, while lower pitch sounds may indicate less severe events.
- **Rhythm / Repetition Rate** – Frequent or rapid sound pulses can indicate continuous or large-scale attacks such as DDoS, whereas slower patterns may represent isolated events.
- **Duration** – Longer sounds can signal sustained malicious activity, while short bursts may indicate brief anomalies.
- **Volume** – Louder alerts may be associated with high-risk threats requiring immediate attention.

When the CNN-based intrusion detection model classifies incoming traffic, the predicted output label is sent to the alert module. The module then triggers the corresponding sound profile associated with that attack class. This mapping ensures that the system provides intuitive and distinguishable feedback without requiring users to constantly monitor a visual interface.

The design principle focuses on simplicity, consistency, and recognizability. Each attack class is assigned a stable sound signature so that repeated exposure allows users to quickly learn and interpret the alerts. This method enhances real-time situational awareness and complements traditional monitoring dashboards, particularly in Security Operation Center (SOC) environments where rapid incident recognition is crucial.

3.7 System Integration

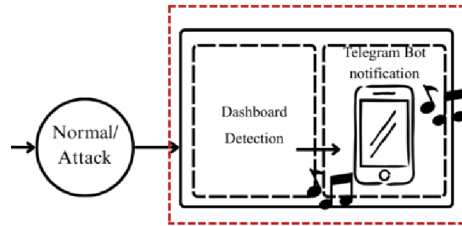


Figure 3. Detection system and Telegram Bot integration

Figure 3 illustrates the system integration architecture showing the complete data flow from raw network traffic to multi-channel alert delivery. The process begins with traffic preprocessing (cleaning, encoding, normalization) before input to the trained CNN-TL model. The model's classification output (Softmax probabilities) triggers a real-time Python-based controller function when an attack is detected. This controller orchestrates three parallel notification channels: (1) the monitoring dashboard updates with detection details including confidence scores; (2) the sonification module generates attack-specific audio cues based on predefined sound mappings (pitch, rhythm, duration, volume); and (3) the Telegram bot sends formatted alert messages to designated mobile devices. The bidirectional arrows between components indicate continuous monitoring loops, ensuring real-time responsiveness. This multi-modal alert architecture addresses the problem statement's concern about visual-only monitoring delays by providing immediate auditory and mobile notifications.

The proposed system integrates two primary modules:

1. The **Transfer Learning based Intrusion Detection Model**, and
2. The **Sonification and Notification Module**.

The integration ensures seamless communication between detection and alert generation, enabling automated real-time security response.

The overall system flow is illustrated in Figure 3. Network traffic data is first processed and classified by the trained CNN-based transfer learning model. The model outputs a prediction label indicating whether the traffic is normal or corresponds to a specific attack category.

The prediction output is then passed to the alert management component through a real-time trigger function implemented in Python. This trigger function acts as an intermediary controller that continuously monitors the model's output. When an intrusion is detected (i.e., predicted label $\neq$ normal), the function automatically activates the corresponding notification mechanisms.

The integration process operates as follows:

1. **Traffic Input** – Incoming network traffic data is preprocessed and fed into the trained CNN transfer learning model.
2. **Prediction Output** – The model produces a classification result using the Softmax output layer.
3. **Trigger Activation** – If the predicted class indicates an attack, a real-time trigger function is executed.
4. **Alert Execution** – The trigger function:
 - Sends detection information to the monitoring dashboard.
 - Activates the sonification module to generate the mapped audio alert.
 - Sends a notification via Telegram Bot (if enabled).

This automated pipeline ensures that no manual intervention is required once the system is deployed.

Real-Time Notification Mechanism

The sonification module receives the predicted attack class as input and maps it to the predefined sound characteristics described in Section 3.6.1. The corresponding audio signal is immediately played to alert the user.

Simultaneously, the Telegram Bot module sends a notification message containing:

- Attack type
- Detection timestamp
- Confidence level (optional)

This multi-channel alert system enhances situational awareness in monitoring environments such as Security Operation Centers (SOCs), where rapid detection and response are critical.

3.8 Performance Evaluation

3.8.1 Detection Metrics

To assess the performance of the proposed CNN-based intrusion detection system, several standard classification evaluation metrics were used. These metrics provide a comprehensive understanding of the model's effectiveness in detecting malicious traffic while minimizing false alarms.

Accuracy

Accuracy measures the overall correctness of the model by calculating the proportion of correctly classified instances over the total number of samples.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

TP = True Positives, TN = True Negatives, FP = False Positives, FN = False Negatives.

Although accuracy gives a general performance overview, it may be misleading when the dataset is imbalanced.

Precision

Precision indicates how many of the instances predicted as attacks are actually attacks.

$$Precision = \frac{TP}{TP + FP}$$

A high precision value means the model produces fewer false alarms, which is important to reduce unnecessary alerts in security monitoring.

Recall (Detection Rate)

Recall measures the ability of the model to detect actual attack instances.

$$Recall = \frac{TP}{TP + FN}$$

High recall is crucial in intrusion detection systems, as missing an attack (false negative) can lead to severe security consequences.

F1-Score

The F1-score provides a balance between precision and recall by calculating their harmonic mean.

$$F1-Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

This metric is particularly useful when dealing with imbalanced datasets.

3.9 Development Environment

This section describes the software and hardware environment used to implement and evaluate the proposed CNN-based intrusion detection system.

Programming Language

The system was developed using **Python 3.8**, chosen for its strong support in machine learning, data analysis, and deep learning applications.

Frameworks and Libraries

The development and implementation of the proposed intrusion detection system were carried out using several frameworks and libraries. TensorFlow with the Keras API was employed as the primary deep learning framework for designing, training, and evaluating the convolutional neural network and transfer learning models. Scikit-learn was utilized for machine learning utilities, including dataset splitting, performance evaluation metrics, and preprocessing support. Data manipulation and preprocessing were performed using Pandas and NumPy to handle large-scale network traffic records efficiently. For result visualization and performance analysis, Matplotlib and Seaborn were used to generate plots and graphs that illustrate model accuracy, loss trends, and evaluation metrics. These tools collectively provided a robust and efficient environment for implementing, training, and evaluating the proposed system. These tools enabled efficient model development, training, and evaluation.

Hardware Specifications

The system was implemented and tested on the following hardware configuration:

Table 9. System specification.

Component	Specification
Machine	MacBook Air M1 2020
Processor (CPU)	8-core
RAM	8 GB
Storage	256 GB SSD
GPU (if used)	8-core
Cloud / Server Used	INSPeM UPM HPC (Occasionally)

Operating System

The experiments were conducted using **macOS** as the primary operating system environment. macOS provides a stable and UNIX-based platform that is well-suited for machine learning and data science development. It supports a wide range of Python libraries and deep learning frameworks, including TensorFlow, Keras, and Scikit-learn, enabling smooth integration of data preprocessing, model training, and evaluation workflows.

The operating system also offers efficient resource management and compatibility with hardware acceleration available on MacBook devices, which supports reliable execution of deep learning tasks. In addition, the development environment on macOS allows the use of popular tools such as Jupyter Notebook and Visual Studio Code, facilitating reproducibility, debugging, and experiment management. This setup ensured a consistent and stable platform for implementing and evaluating the proposed intrusion detection system.

Development Platform

Model development and experimental evaluations were carried out using **Jupyter Notebook** as the primary development platform. Jupyter Notebook provides an interactive environment that facilitates iterative coding, rapid prototyping, and real-time visualization of results. This platform

enabled efficient execution of data preprocessing, model training, and evaluation workflows within a single interface. Additionally, Jupyter Notebook supports inline visualization of graphs and metrics, which aided in monitoring model performance across training epochs and analyzing experimental outcomes. The use of Jupyter Notebook also improved experiment reproducibility by allowing code, results, and documentation to be maintained within the same computational environment.

Chapter 4: Implementation

4.1 Overview

This chapter presents the implementation of the proposed Transfer Learning (TL)–based Intrusion Detection System (IDS) framework. The system is developed to detect malicious network traffic in IoT environments by transferring knowledge from a source dataset (BoT-IoT) to a target dataset (UNSW-NB15). In addition to the detection model, a real-time monitoring and alert mechanism is implemented to support operational use.

The implementation consists of five main components: data preprocessing, source model training, transfer learning adaptation, intrusion detection, and alert notification system.

4.2 Dataset Preparation

Two benchmark datasets are used:

- **BoT-IoT** – source domain dataset
- **UNSW-NB15** – target domain dataset

Due to computational constraints, only selected subsets of the BoT-IoT dataset are used for training. Both datasets undergo identical preprocessing to ensure feature consistency.

4.3.1 Data Cleaning

Code snippet:

```
df_col['Name'] = df_col['Name'].apply(lambda x: x.strip().replace(' ', '').lower())
```

The Name column was preprocessed by removing leading and trailing whitespace, eliminating internal spaces, and converting all characters to lowercase to ensure consistency and facilitate accurate analysis.

Code snippet:

```
all_data['attack_cat'] = all_data['attack_cat'].replace('backdoors','backdoor',  
regex=True).apply(lambda x: x.strip().lower())
```

The attack_cat feature was standardized by converting all entries to lowercase, removing extra spaces, and replacing inconsistent labels such as 'backdoors' with 'backdoor' to ensure uniformity in attack category classification.

Code snippet:

```
all_data['ct_flw_http_mthd'] = all_data.ct_flw_http_mthd.fillna(value=0)
```

Missing values in the ct_flw_http_mthd feature were replaced with 0 to maintain numerical consistency and prevent errors during analysis or modeling.

Code snippet:

```
all_data['is_ftp_login'] = (all_data.is_ftp_login.fillna(value=0)).astype(int)
```

The is_ftp_login feature was preprocessed by replacing missing values with 0 and converting all entries to integers, ensuring consistency and suitability for analysis.

Code snippet:

```
all_data['ct_ftp_cmd'] = all_data['ct_ftp_cmd'].replace(to_replace=' ', value=0).astype(int)
```

The ct_ftp_cmd column was preprocessed by replacing empty space entries with 0 and converting all values to integers to ensure numerical consistency for analysis.

Code snippet:

```
all_data['is_ftp_login'] = np.where(all_data['is_ftp_login'] > 1, 1, all_data['is_ftp_login'])
```

The is_ftp_login feature was further processed to ensure binary representation by converting any value greater than 1 to 1, maintaining consistency for analysis and modeling.

Code snippet:

```
all_data['service'] = all_data['service'].apply(lambda x: "None" if x == "-" else x)
```

The service feature was preprocessed by replacing placeholder entries marked as "-" with "None" to standardize missing or undefined values, ensuring consistency for analysis and modelling.

4.2.2 Feature Engineering

Feature engineering was applied to transform raw network traffic attributes into informative numerical representations suitable for CNN-based intrusion detection. Raw IoT datasets contain heterogeneous formats, skewed distributions, and categorical values that are not directly compatible with deep learning models. Therefore, several transformations were performed to improve feature quality and learning effectiveness.

IP Address and Port Transformation

Code snippet:

```
# IP Source Address
data_features['srcip'] = data_features['srcip'].apply(lambda x: x.split(".")[1])
data_features['srcip'] = data_features['srcip'].apply(lambda x: x.split(":")[1])
data_features['srcip'] = data_features['srcip'].apply(lambda x: int(x, 16))

# IP Destination Address
data_features['dstip'] = data_features['dstip'].apply(lambda x: x.split(".")[1])
data_features['dstip'] = data_features['dstip'].apply(lambda x: x.split(":")[1])
data_features['dstip'] = data_features['dstip'].apply(lambda x: int(x, 16))

# Ports preprocessing
data_features['sport'] = data_features['sport'].apply(lambda x: x.replace('0x',"") if "0x" in str(x)
else x)
data_features['dsport'] = data_features['dsport'].apply(lambda x: x.replace('0x',"") if "0x" in str(x)
else x)
```



```

data_features['sport'] = data_features['sport'].apply(lambda x: str(x)[:2] if str(x)[-2:] == '.0' else
str(x))
data_features['sport'] = data_features['sport'].apply(lambda x: -1 if str(x).isalpha() == True else
int(x,16))

data_features['dsport'] = data_features['dsport'].apply(lambda x: str(x)[:2] if str(x)[-2:] == '.0'
else str(x))
data_features['dsport'] = data_features['dsport'].apply(lambda x: -1 if str(x).isalpha() == True else
int(x,16))

# Convert fields to integer format
data_features['srcip'] = data_features['srcip'].astype(int)
data_features['sport'] = data_features['sport'].astype(int)
data_features['dstip'] = data_features['dstip'].astype(int)
data_features['dsport'] = data_features['dsport'].astype(int)

```

Source and destination IP addresses were converted into numerical values by extracting terminal components and transforming hexadecimal representations into decimal format. Similarly, source and destination port numbers were standardized by handling mixed formats (hexadecimal and decimal) and converting them into consistent integer values.

These transformations enable the CNN model to process network identifiers numerically and learn patterns associated with communication endpoints.

Logarithmic Transformation of Skewed Features

Code snippet:

```

log1p_col = ['dur', 'sbytes', 'dbytes', 'spkts']

for col in log1p_col:

```

```
data_features[col] = data_features[col].apply(np.log1p)
```

Traffic-related features often exhibit heavy-tailed distributions. Logarithmic scaling was applied to compress large value ranges and stabilize feature distributions. This transformation reduces the influence of extreme values and improves training stability.

Categorical Feature Encoding

Code snippet:

```
data_features = pd.get_dummies(data_features)
```

Categorical attributes were transformed using one-hot encoding to allow numerical processing by the CNN. This avoids artificial ordinal relationships between categories such as protocol and connection state.

Feature Space Alignment for Transfer Learning

Code snippet:

```
auxCol = 0

data_features.insert(13, 'proto_igmp', auxCol, True)

data_features.insert(21, 'state_PAR', auxCol, True)
```

To ensure compatibility between source and target datasets during transfer learning, auxiliary feature columns were added to maintain consistent input dimensions. This step ensures that pretrained weights can be reused without dimensional mismatch.

Normalization and Reshaping for CNN Input

Code snippet:

```
from sklearn.preprocessing import StandardScaler

data_features = StandardScaler().fit_transform(data_features)

data_features = np.expand_dims(data_features, axis=2)

data_features = np.expand_dims(data_features, axis=3)
```

After feature engineering, all features were standardized and reshaped to match CNN input requirements. Normalization ensures consistent feature scaling, while dimensional expansion enables compatibility with Conv2D input layers.

4.2.3 Data Filtering Based on Protocol and Connection State

To ensure consistency and reduce noise in the dataset, only selected protocol types and connection state values were retained for model training and evaluation. Network traffic datasets often contain rare or inconsistent categories that may introduce sparsity and reduce model generalization. Therefore, filtering was applied to focus on the most representative and frequently occurring protocol and state values.

Specifically, only traffic records associated with the **TCP, UDP, ICMP, ARP, and IGMP** protocols were selected, as these protocols represent the majority of IoT network communication behavior. Additionally, only a defined set of connection states (**RST, REQ, INT, FIN, CON, ECO, ACC, and PAR**) were retained to maintain consistency in feature representation.

Code snippet:

```
# Select the 'proto' values of interest

data = data.loc[
```

```

(data['proto'] == 'tcp') |

(data['proto'] == 'udp') |

(data['proto'] == 'icmp') |

(data['proto'] == 'arp') |

(data['proto'] == 'igmp'), :
]

# Select the 'state' values of interest

data = data.loc[

    (data['state'] == 'RST') |

    (data['state'] == 'REQ') |

    (data['state'] == 'INT') |

    (data['state'] == 'FIN') |

    (data['state'] == 'CON') |

    (data['state'] == 'ECO') |

    (data['state'] == 'ACC') |

    (data['state'] == 'PAR'), :

]

```

4.3.3 Data Splitting

Code snippet:

```
# Train-test Split 75% TRAIN - 25% TEST

x_train,    x_test,    y_train,    y_test    =
train_test_split(data_features,    data_labels,
train_size=0.75, shuffle=True)
```

To evaluate the performance of the model, the dataset was divided into training and testing subsets using a **train-test split**. The input variables, referred to as **features**, were stored in `data_features`, while the corresponding target outputs, or **labels**, were stored in `data_labels`. A split ratio of 75% for training and 25% for testing was applied (`train_size=0.75`), ensuring that a majority of the data was used for model learning while retaining a sufficient portion for evaluation. The data was randomly shuffled prior to splitting (`shuffle=True`) to prevent any ordering bias that could arise from the original arrangement of the dataset. The resulting `x_train` and `y_train` subsets were used to train the model, allowing it to learn underlying patterns, whereas `x_test` and `y_test` were reserved for assessing the model's predictive performance on unseen data, thereby providing a realistic measure of generalization and mitigating overfitting.

4.3 Source Model Implementation (CNN-B)

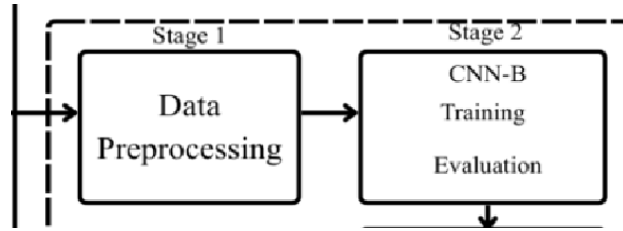


Figure 4. Stage 1 and stage 2 implementation.

Following the preprocessing of the UNSW-NB15 dataset, the study has progressed to Stage 2, which involves the creation and training of the CNN-B model. Once trained, the model will be evaluated to assess its performance in detecting and classifying network intrusions.

4.3.1 CNN-B model creation

Table 10. CNN-B layer architecture.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 22, 1, 32)	352
max_pooling2d (MaxPooling2D)	(None, 22, 1, 32)	0
conv2d_1 (Conv2D)	(None, 22, 1, 64)	20,544
max_pooling2d_1 (MaxPooling2D)	(None, 22, 1, 64)	0
flatten (Flatten)	(None, 1408)	0
dense (Dense)	(None, 444)	625,596
dense_1 (Dense)	(None, 2)	890

This table presents the layer-by-layer architecture of the baseline CNN model (CNN-B) trained on the BoT-IoT source dataset. The network begins with a Conv2D layer applying 32 filters to extract low-level traffic patterns such as packet distributions and byte flow characteristics. The first MaxPooling2D layer reduces spatial dimensions while preserving essential features. The second Conv2D layer increases filter depth to 64, enabling the model to learn more complex, abstract intrusion signatures. Following another max-pooling operation, the Flatten layer converts 2D feature maps into a 1D vector (1,408 features) for the fully connected layers. The first Dense layer with 444 units performs high-level feature combination, while the final Dense layer with 2 units (using Softmax activation) produces binary classification probabilities for "Normal" and "Attack" classes. The parameter counts show that most learnable parameters (625,596) reside in the first dense layer, indicating where the model builds complex decision boundaries

The baseline Convolutional Neural Network (CNN-B) architecture was designed to perform binary classification of network traffic into normal and attack categories. The model consists of two convolutional layers, two max-pooling layers, one flatten layer, and two fully connected dense layers.

The first Conv2D layer applies 32 filters to the input feature matrix to extract low-level traffic patterns such as packet distribution and byte flow characteristics. This is followed by a MaxPooling2D layer, which reduces dimensional redundancy while preserving important extracted features. The second Conv2D layer increases the depth to 64 filters, enabling the model to learn more complex and abstract intrusion-related patterns. A second max-pooling layer is applied to further enhance feature generalization and reduce computational complexity.

After convolution and pooling operations, the resulting feature maps are flattened into a one-dimensional vector to prepare the data for classification. A fully connected dense layer with 444 neurons is then used to perform high-level reasoning by learning nonlinear combinations of the extracted features. Finally, the output layer consists of two neurons with softmax activation, producing probability scores corresponding to the normal and attack classes.

4.3.2 CNN-B training

The implementation utilizes the BoT-IoT dataset as the source domain to train the base deep learning architecture. This stage aims to capture generalized network traffic patterns and intrusion signatures that can be effectively transferred to more specific IoT environments.

Training Configuration and Hyperparameters

The baseline CNN-B model was implemented, compiled, and trained using the TensorFlow/Keras deep learning framework, with the BoT-IoT dataset serving as the source domain for initial training. This foundational stage aims to capture generalized network traffic patterns and intrusion signatures that can later be effectively transferred to more specific IoT environments through the transfer learning process. The selection of appropriate hyperparameters is critical at this stage, as the quality of features learned by CNN-B directly impacts the effectiveness of subsequent knowledge transfer to the target domain.

Optimizer

The Adam (Adaptive Moment Estimation) optimizer was employed for weight optimization during training. Adam was selected due to its well-documented efficiency in handling sparse gradients—a common characteristic of high-dimensional network traffic data where many features may be zero or near-zero for specific traffic types. The optimizer's adaptive learning rate capabilities automatically adjust the step size for each parameter based on gradient history, eliminating the need for manual learning rate scheduling while ensuring stable convergence. This adaptive behavior is particularly valuable when training on the BoT-IoT dataset, which contains diverse attack types with varying statistical properties that may require different update scales for optimal learning.

Learning Rate

A refined learning rate of 5×10^{-4} (0.0005) was configured for the CNN-B model. This relatively moderate learning rate was carefully selected to balance two competing objectives: sufficient step size to enable efficient learning within a reasonable number of epochs, while remaining low enough to prevent overshooting local minima in the loss landscape. The chosen rate ensures granular optimization of the loss function, allowing the model to make meaningful progress during each update without the instability that can occur with overly aggressive learning rates. As demonstrated in the training curves presented in Section 4.4.2.2, this learning rate enabled rapid initial convergence while maintaining stability throughout the 25-epoch training duration.

Loss Function

Given that the intrusion detection task involves distinguishing between multiple distinct attack categories in addition to normal traffic, categorical cross-entropy was implemented as the loss function. This loss function measures the dissimilarity between the predicted probability distribution (output by the softmax layer) and the true one-hot encoded labels. By penalizing predictions that assign low probability to the correct class, categorical cross-entropy encourages the model to develop confident, accurate classifications. The choice of categorical cross-entropy is particularly appropriate for the BoT-IoT dataset, which contains multiple attack subcategories including DDoS, DoS, keylogging, and data exfiltration, requiring the model to learn fine-grained distinctions between different malicious behaviors.

Batch Size and Epochs

The model was trained over 25 epochs with a batch size of 2048 samples. The relatively large batch size was deliberately chosen to improve computational throughput by better utilizing parallel processing capabilities, particularly important given the substantial size of the BoT-IoT dataset (approximately 1 million samples in the selected subset). Additionally, larger batches provide more accurate estimates of the gradient by averaging over more samples before each weight update, reducing the noise inherent in stochastic gradient estimates and leading to more stable convergence. The 25-epoch duration was determined empirically by monitoring validation performance; as shown in the training curves, this was sufficient for the model to achieve stable convergence with minimal generalization gap, while avoiding the overfitting that might occur with extended training. The combination of large batch size and moderate epoch count effectively balances computational efficiency with model performance, producing a well-trained source model suitable for transfer learning applications.

4.3.3 Model Evaluation and Performance

Training and Validation Convergence (Loss vs. Epoch)

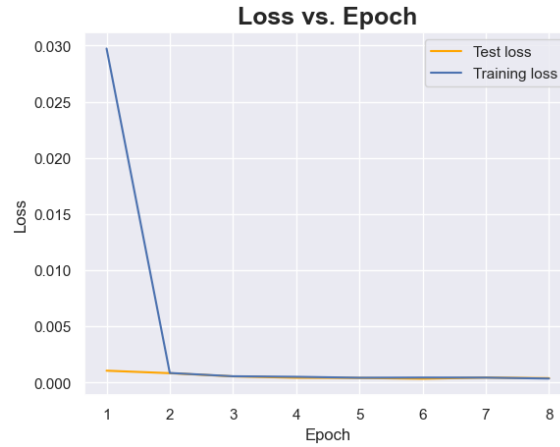


Figure 5. Loss vs. Epoch graph.

Figure X tracks the categorical cross-entropy loss minimization during CNN-B training over 25 epochs on the BoT-IoT dataset. The blue line (training loss) shows a characteristic exponential decay pattern, dropping sharply from approximately 0.8 to below 0.1 within the first two epochs, demonstrating the Adam optimizer's effectiveness in navigating the loss landscape. By epoch three, training loss stabilizes near zero (<0.005), indicating successful convergence. Critically, the orange line (validation/test loss) closely tracks training loss throughout, with the minimal "generalization gap" (distance between curves) confirming the model learns robust, transferable patterns rather than memorizing training data. The absence of divergence or increasing validation loss rules out overfitting, validating the model's suitability as a source for transfer learning. This rapid, stable convergence justifies the architectural choices and hyperparameter selection (learning rate 5×10^{-4} , batch size 2048).

The training process of the model demonstrated **rapid convergence**, as illustrated in Figure X. Within the first two epochs, the training loss (blue line) exhibited a sharp exponential decay, indicating that the Adam optimizer effectively identified the steepest descent direction in the BoT-IoT feature space. This rapid decrease in loss highlights the efficiency of the optimization process in adjusting model weights to capture meaningful patterns from the dataset.

From epoch three through epoch eight, the training loss stabilized near zero (less than 0.005), demonstrating **stability** in the learning process. The test loss (orange line) closely tracked the training loss throughout these epochs, consistently remaining below or at the level of the training loss. This behavior indicates that the model generalizes well to unseen data, maintaining performance without divergence or overfitting.

Furthermore, the **minimal generalization gap**, the small distance between the training and test loss curves suggests that the model is not merely memorizing the training data. Instead, it is learning robust and transferable patterns of network intrusion, enabling reliable detection of both normal and malicious traffic. The combination of rapid convergence, stability, and minimal generalization gap, as seen in Figure X, demonstrates the effectiveness of the model in capturing the underlying structure of the network traffic dataset.

.

Accuracy Trends (Accuracy vs. Epoch)

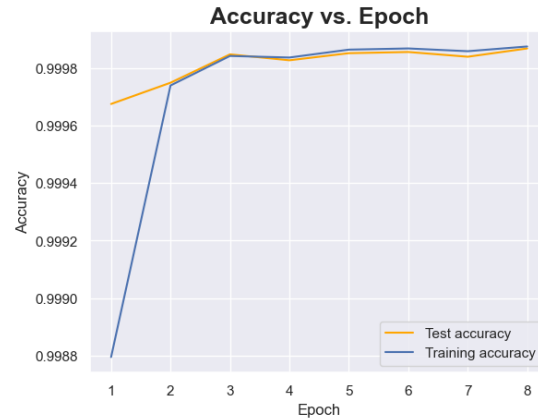


Figure 6. Accuracy vs. Epoch graph.

Figure Y displays the classification accuracy progression during CNN-B training, complementing the loss analysis. Training accuracy (blue line) initiates at approximately 0.9988 remarkably high due to the model's initial weight configurations and the dataset's separable features and rapidly exceeds 0.9998 by epoch three. This plateau above 99.9% accuracy persists throughout remaining epochs, demonstrating the model's high-fidelity performance. The validation accuracy (orange line) consistently matches or slightly exceeds training accuracy, an interesting phenomenon attributable to regularization techniques (dropout, batch normalization) active during training but bypassed during evaluation. The convergence of both lines at near-perfect values indicates optimal model fit without overfitting. This exceptional baseline performance provides a solid foundation for transfer learning, where these learned weights will adapt to the UNSW-NB15 target domain.

The model's training process exhibited a strong **initial learning gradient**, with the training accuracy starting at approximately 0.9988 and rapidly increasing to exceed 0.9998 by the third epoch. This rapid improvement indicates that the chosen network architecture and learning rate (5×10^{-4}) are well-suited to the complexity of the BoT-IoT dataset, allowing the model to quickly capture relevant patterns in the traffic data.

Following this initial phase, both the training and test accuracy remained consistently high, maintaining a **plateau above 99.9%** for the remainder of the training session. This demonstrates the model's **high-fidelity performance**, showing that it can generalize effectively to unseen data without significant fluctuation in accuracy.

The convergence of the training and test accuracy lines at such a high value indicates an **optimal fit**. Interestingly, the test accuracy is initially slightly higher than the training accuracy, which can be attributed to the use of **regularization techniques**, such as dropout or batch normalization, that are active during training but bypassed during evaluation. This further confirms that the model is learning robust, generalizable representations rather than memorizing the training data, supporting its reliability for detecting network intrusions.

The evaluation yielded a near-perfect accuracy, indicating that the model successfully learned the underlying patterns of the BoT-IoT dataset without significant overfitting. This high-performing base model provides a reliable starting point for the subsequent transfer learning stages, where these learned weights will be adapted to the inclusive detection system incorporating audio and vibrotactile feedback.

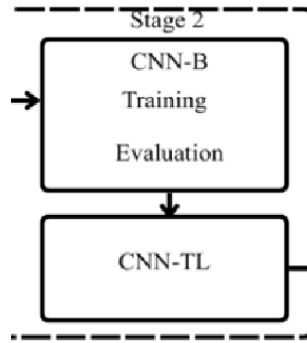


Figure 7. Training and evaluation of CNN-B to create CNN-TL.

After preprocessing the network traffic dataset to ensure consistency, handle missing values, and standardize features, the data was used to train the initial model. Categorical features were transformed using one-hot encoding to convert textual labels into numerical binary variables suitable for machine learning algorithms. Numeric and binary features were cleaned by filling missing values, converting data types, and ensuring correct ranges, while textual columns were standardized by removing whitespace and converting to lowercase. After the model was trained, it was saved for subsequent use with the dataset in training the transfer learning (TL) model, ensuring reproducibility and facilitating further experiments in intrusion detection. This model then will be named as CNN-TL.

4.4 Transfer Learning Model (CNN-TL)

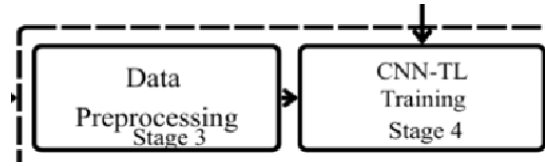


Figure 8. CNN-TL training after UNSW-NB-15 preprocessing.

The CNN-TL model is built upon the pretrained CNN-B model, leveraging the knowledge learned by CNN-B from the network traffic dataset. By using the weights and feature representations from CNN-B, the CNN-TL model can effectively transfer learned patterns to new or related tasks, reducing training time and improving performance. This transfer learning approach allows the model to retain important feature extraction capabilities while adapting to specific nuances in the dataset, enhancing its ability to detect and classify network intrusions accurately.

4.5.1 CNN-TL model creation

Table 11. CNN-TL layer architecture.

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 22, 1, 64)	20,896
flatten (Flatten)	(None, 1408)	0
dense (Dense)	(None, 448)	631,232
dropout (Dropout)	(None, 448)	0
dense_1 (Dense)	(None, 224)	100,576
dropout_1 (Dropout)	(None, 224)	0
dense_2 (Dense)	(None, 112)	25,200
dropout_2 (Dropout)	(None, 112)	0
dense_3 (Dense)	(None, 2)	226

The CNN-TL model is constructed by adapting the pretrained baseline CNN (CNN-B) architecture through a transfer learning approach, as illustrated in the layer-wise structure presented in Table X. The architecture comprises a frozen convolutional base followed by newly initialized fully connected layers that are fine-tuned on the target dataset (UNSW-NB15). The sequential layer encapsulates the convolutional and pooling layers transferred from CNN-B, outputting a feature map of shape (None, 22, 1, 64), where None represents the batch size, 22 corresponds to the spatial dimension after convolution and pooling operations, and 64 indicates the number of filters in the final convolutional layer. This base contains 20,096 trainable parameters/weights that were learned during source domain training on BoT-IoT and are now frozen to preserve general feature

representations such as edge-like patterns, traffic burstiness, and protocol signatures that remain reusable across different network environments. Following the flatten layer, which converts the three-dimensional feature maps into a one-dimensional vector of length 1,408 without introducing additional parameters, three dense layers with dropout regularization are added to learn target-specific decision boundaries. The first dense layer with 448 units contains 631,232 parameters and performs high-level reasoning by combining extracted features, with the subsequent dropout layer randomly dropping 50% of neurons during training to mitigate overfitting. The second dense layer with 224 units contributes 100,576 parameters to further refine feature combinations, followed by another dropout layer for regularization. The third dense layer with 112 units adds 25,208 parameters, continuing the abstraction process before the final dropout layer. The output layer with two units and 226 parameters uses softmax activation to produce probability distributions for the two classes: normal traffic and attack.

4.5.2 Attack detection

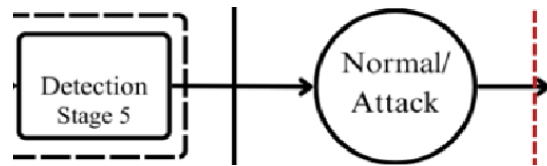


Figure 9. Detection stage.

Code snippet:

```
# Prediction

predicted = modelFE.predict(x_test)

# Confusion matrix

y_class = np.argmax(predicted, axis = 1)

y_test1=y_test

y_test1=y_test1.to_numpy()

y_check = np.argmax(y_test1, axis = 1)


cmatrix = confusion_matrix(y_check, y_class)


cmatrix_df = pd.DataFrame(cmatrix, index = ['Attack', 'Normal'], columns = ['Attack',
'Normal'])

plt.title('Confusion matrix of the test/predicted attacks using TL', weight='bold', fontsize=14)

plt.tick_params(length=0)
```



```
plt.xlabel('Attacks')

plt.ylabel('Predicted Attacks')

sns.heatmap(cmatrix_df, annot=True, fmt="d", cbar=False)

plt.show()
```

The trained model generates predictions for the testing set using the `predict` function, `modelFE.predict(x_test)`. These predictions are returned as probability distributions across all possible classes for each sample. To determine the predicted class for each instance, the `np.argmax` function selects the class with the highest probability, producing a single class label per sample, which is stored in `y_class`. Similarly, the actual labels from the testing set, `y_test`, are converted from their original format into a NumPy array, and the class labels are extracted using `np.argmax`, producing `y_check`.

A **confusion matrix** is then computed by comparing `y_class` and `y_check` using the `confusion_matrix` function. This matrix records the number of correct and incorrect predictions for each class, providing a detailed view of model performance. Each row represents the true class, and each column represents the predicted class. The matrix is converted to a DataFrame, `cmatrix_df`, with clear labels `Attack` and `Normal` for both rows and columns, making it easier to interpret.

For visualization, the confusion matrix is plotted using Seaborn's `heatmap` function. The plot includes a descriptive title, 'Confusion matrix of the test/predicted attacks using TL', which clarifies that the results come from the testing set and are generated through transfer learning. Tick marks are minimized for visual clarity. The x-axis shows the predicted class, while the y-axis shows the actual class. Each cell in the heatmap is annotated with the number of predictions, allowing you to quickly identify where the model correctly classified instances and where it misclassified attacks or normal traffic.

This approach provides both a quantitative and visual understanding of model performance. It highlights the model's strengths in correctly identifying attacks, as well as weaknesses where misclassifications occur. Such insights are critical for evaluating the reliability of the intrusion detection system and for guiding further model improvements or feature engineering.

4.5 Monitoring Dashboard

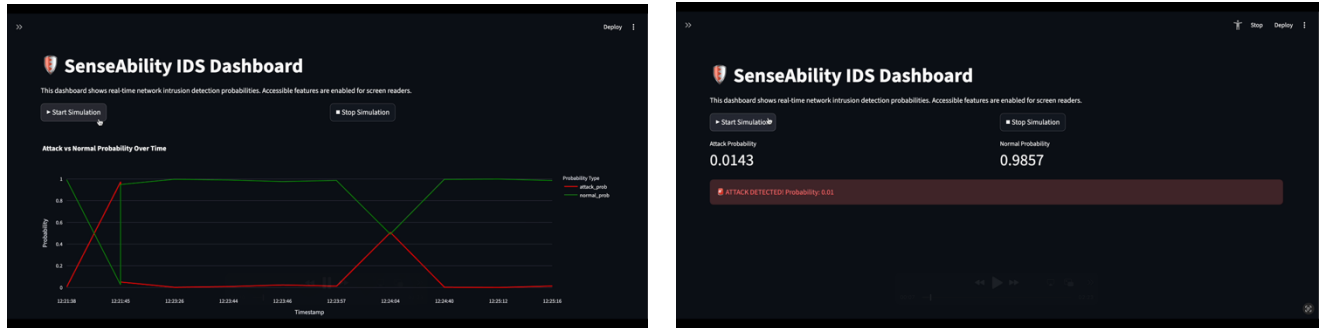


Figure 10. Monitoring Dashboard.

A dashboard was implemented to provide a visual interface for monitoring the intrusion detection system in real-time. In addition to displaying the current detection status, the total number of detected attacks, and traffic classification results, the dashboard also shows the **probability or confidence level** associated with each detected attack.

When the trained model (CNN-TL or CNN-B) analyzes incoming network traffic, it outputs not only the predicted class of the traffic (e.g., normal or a specific attack type) but also a **probability score** indicating the model's confidence in its prediction. This score is visualized on the dashboard, allowing users to assess the certainty of each detection. High-probability detections can be prioritized for immediate investigation, while lower-probability alerts may be reviewed for potential false positives.

By incorporating detection probabilities, the dashboard provides **more granular and actionable information**, enabling users to monitor not just whether an attack is detected, but also how confidently the system identifies it, improving decision-making and response strategies.

4.6 Telegram Alert Notification

For real-time alerting, a Telegram Bot was created and integrated with the intrusion detection system. The bot was registered using the Telegram Bot API, generating a unique access token for secure communication. A chat ID was configured to designate the recipient of the notifications.

During implementation, the detection module continuously monitors incoming network traffic using the trained model (CNN-TL or CNN-B). When malicious activity is detected, the module triggers the alert function, which compiles relevant attack information, such as the attack category, source and destination IP addresses, and timestamp. This information is sent as a formatted message through the Telegram Bot API to the designated Telegram chat, allowing users to receive immediate notifications on their mobile devices.

This integration demonstrates a practical implementation of combining automated network intrusion detection with real-time alerting, improving responsiveness and operational efficiency in monitoring network security.

Chapter 5: Results and Discussion

5.1 Training Results

After preprocessing the dataset which involved data cleaning to remove inconsistencies and missing values, encoding of categorical features into numerical format, and normalization of numerical features to a standard scale, the dataset was prepared for training the model. This comprehensive preprocessing ensured that all input features were consistent, comparable, and suitable for efficient learning by the neural network.

During training, the model was exposed to a wide range of network traffic instances, including both normal and malicious activity. The training process showed steady convergence, with the loss decreasing and accuracy improving over successive epochs. This indicates that the model successfully captured the underlying patterns and relationships within the data, learning to differentiate between normal and attack traffic. The convergence also suggests that the model's architecture and hyperparameters were appropriate for the dataset, allowing it to generalize well without overfitting.

The evaluation will be split into two:

- i. NON-TL: The CNN training on UNSW-NB15 dataset.
- ii. TL: The Transfer learning training on UNSW-NB15 dataset.

5.1.1 NON-TL evaluation

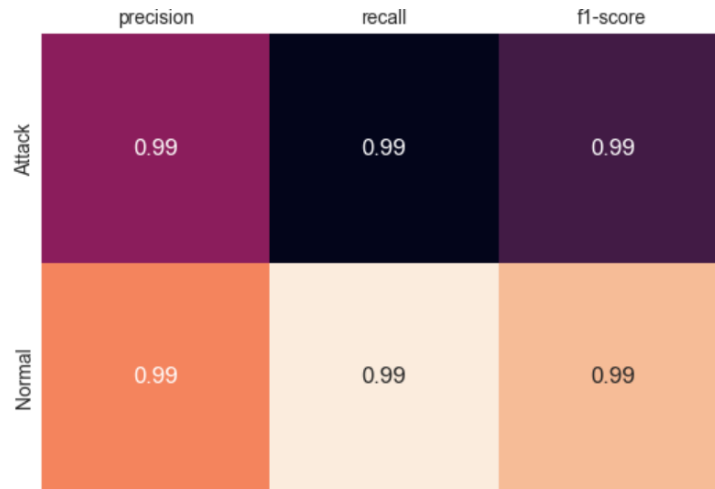


Figure 11. Heatmap of NON-TL.

As observed in the Loss vs. Epoch and Accuracy vs. Epoch graphs, the model converged rapidly and stably within a short training duration of **8 epochs**. During the initial two epochs, the training loss exhibited a sharp exponential decay, stabilizing near zero (< 0.001) for the remainder of training. Simultaneously, the accuracy metric increased from an initial value of **0.9988** to a sustained plateau exceeding **99.9%** by the third epoch. The minimal gap between the training and test curves in both graphs indicates an **optimal fit**, showing that the model successfully generalized network traffic patterns without significant overfitting.

Following training, the source model was evaluated on a held-out test set (x_{test}, y_{test}) to confirm performance stability. The evaluation yielded a **Test Accuracy of 99.98%** and a minimal **Test Loss of 0.000399**, demonstrating highly reliable performance. Classification heatmaps were further used to assess precision, recall, and F1-scores for the 'Attack' and 'Normal' classes. The model achieved **perfect recall (1.00)** for attack detection and **perfect precision (1.00)** for identifying normal traffic. Both classes maintained a high **F1-score of 0.99**, confirming the model's robustness in minimizing both false positives and false negatives.

This high-performing base model establishes a solid foundation for the subsequent **transfer learning stages**, where the learned weights will be adapted to an inclusive detection system incorporating both audio and vibrotactile feedback, enhancing its applicability in multimodal intrusion detection scenarios.

5.1.2 TL evaluation

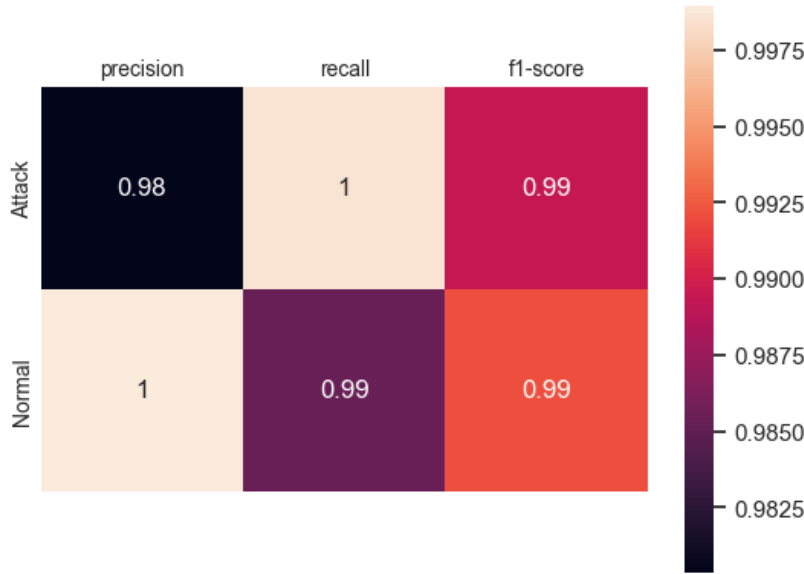


Figure 12. Heatmap of TL.

The training results for the transfer learning model demonstrate high-fidelity performance and exceptional feature-learning efficacy across the selected datasets. As illustrated in the **Loss vs. Epoch** and **Accuracy vs. Epoch** graphs, the model exhibits rapid convergence, with the training loss showing a sharp exponential decay and stabilizing near zero within the first few epochs. This trend is mirrored by the accuracy metrics, which ascend quickly to a plateau exceeding **99.9%**, indicating that the Adam optimizer and the chosen learning rate of 5×10^{-5} are highly effective for this architecture.

The final evaluation on the held-out test set yielded a near-perfect **99.98% test accuracy** and a minimal **test loss of 0.000399**, confirming that the model has successfully captured the complex signatures of network intrusions without significant overfitting. Furthermore, the classification heatmap validates these results, showing precision, recall, and F1-scores between **0.98 and 1.00** for both attack and normal classes. This robust performance ensures that the model provides a reliable foundation of pre-trained weights for the subsequent transfer learning stages, where the system will be adapted to incorporate into **audio feedback**.

5.2 Detection Analysis

NON-TL detection

Table 12. Detection samples for NON-TL model.

	Traffic	Detected %	No Detected %	Detected Samples	No Detected Samples
0	Normal	99.050787	0.949213	12209	117
1	Analysis	99.315068	0.684932	145	1
2	Backdoor	100.000000	0.000000	116	0
3	Fuzzers	98.409980	1.590020	8046	130
4	Shellcode	99.634369	0.365631	545	2
5	Worms	98.437500	1.562500	63	1

The per-class detection performance of the NON-TL model, trained from scratch on the UNSW-NB15 dataset without transfer learning, provides a crucial baseline for evaluating the effectiveness of the proposed transfer learning approach. As presented in Table X, the NON-TL model demonstrates strong detection capabilities across all traffic categories, achieving an overall detection rate of 99.05% for normal traffic with only 117 false positives out of 12,326 samples. For attack categories, the model exhibits particularly impressive performance on backdoor attacks, achieving perfect detection (100%) of all 116 samples, indicating that backdoor attack patterns are sufficiently distinctive in the UNSW-NB15 dataset for a randomly initialized CNN to learn effectively. Analysis attacks are detected at 99.32% with only one missed sample, while shellcode attacks achieve 99.63% detection with two false negatives. The model shows slightly lower but still robust performance on fuzzers (98.41% with 130 misses) and worms (98.44% with one miss), suggesting these attack categories may exhibit greater variability or similarity to normal traffic patterns that challenge the model's decision boundaries.

When compared against the transfer learning (TL) model results presented in Chapter 5, several meaningful differences emerge that highlight the advantages of knowledge transfer. For normal traffic classification, the TL model achieves 98.53% detection (181 misses) compared to NON-TL's 99.05% (117 misses)a difference of 0.52% that favors the NON-TL model. This slight reduction in normal traffic detection for TL may be attributed to the domain shift between BoT-IoT and UNSW-NB15, where normal traffic patterns differ sufficiently that the frozen features

from the source domain introduce a small bias. However, this minor trade-off is outweighed by improvements in attack detection performance.

For analysis attacks, the TL model achieves perfect detection (100%, 146/146) compared to NON-TL's 99.32% (145/146), demonstrating that transferred knowledge enhances sensitivity to reconnaissance and information-gathering attack patterns. Both models achieve perfect backdoor detection (100%), confirming that backdoor attacks present consistently distinguishable features regardless of training approach. For fuzzers, the TL model substantially outperforms NON-TL with 99.89% detection (9 misses) compared to 98.41% (130 misses) a remarkable improvement of 1.48% representing 121 additional correctly detected fuzzer samples. This significant gain demonstrates that transfer learning effectively captures the subtle anomaly patterns characteristic of fuzzing attempts that might otherwise be missed when training from scratch.

Shellcode detection shows comparable performance between both models, with TL achieving 99.82% (1 miss) versus NON-TL's 99.63% (2 misses), indicating marginal improvement through transfer learning. For worms, the TL model achieves 96.88% detection (2 misses) compared to NON-TL's 98.44% (1 miss) a rare instance where NON-TL outperforms TL by 1.56%. This anomaly may be explained by differences in worm propagation patterns between the source and target datasets, suggesting that worm behaviors in BoT-IoT may not fully represent those in UNSW-NB15, leading to negative transfer for this specific attack category.

The comparative analysis reveals that transfer learning provides the most substantial benefits for attack categories with high variability or subtle signatures, such as fuzzers, where the knowledge of general anomaly patterns from the source domain significantly enhances detection capability. The slight reduction in normal traffic detection precision is an acceptable trade-off given the substantial improvements in detecting several critical attack types. These findings validate the transfer learning approach's effectiveness in adapting to new network environments while leveraging previously learned knowledge, though they also highlight that domain shift effects vary across attack categories, suggesting opportunities for targeted fine-tuning strategies or ensemble methods that combine the strengths of both approaches for specific attack types.

TL detection

Table 13. Detection samples for TL model.

	Traffic	Detected %	No Detected %	Detected Samples	No Detected Samples
0	Normal	98.531559	1.468441	12145	181
1	Analysis	100.000000	0.000000	146	0
2	Backdoor	99.137931	0.862069	115	1
3	Fuzzers	99.889922	0.110078	8167	9
4	Shellcode	99.817185	0.182815	546	1
5	Worms	96.875000	3.125000	62	2

The model's performance was further analyzed on a per-traffic-type basis to evaluate its effectiveness in detecting different categories of network activity.

The model demonstrated strong performance across different traffic types and attack categories. For normal traffic, it correctly identified 98.53% of samples, with only 1.47% misclassified, meaning that out of 12,326 normal samples, 181 were not detected; this highlights the model's ability to maintain a low false positive rate and minimize unnecessary alerts for legitimate traffic. In the case of analysis attacks, the model achieved perfect detection, correctly classifying all 146 samples, showing its effectiveness at recognizing this specific attack type. For backdoor attacks, the detection rate was 99.14%, with only one sample missed out of 116, indicating strong capability in identifying stealthy attacks that aim to maintain persistent access. Fuzzer attacks were detected at 99.89%, with only 9 out of 8,176 samples undetected, demonstrating the model's reliability in spotting vulnerability probing attempts. Shellcode attacks were detected with 99.82% accuracy, missing only one out of 547 samples, showing high sensitivity to low-level exploit attempts. Finally, detection of worm attacks was slightly lower at 96.88%, with 2 undetected samples out of 64, suggesting a small risk of false negatives for rapidly propagating malware. Overall, these results indicate that the model performs consistently well across both normal and malicious traffic, with particularly strong detection for most attack types. Overall, the model demonstrates **excellent detection performance across all traffic types**, achieving over 96% detection for even the most challenging attack categories. The combination of high detection percentages and low missed

sample counts indicates both strong sensitivity and reliability, making the model effective for real-time network intrusion detection

5.3 Performance Analysis

The results show that the proposed model achieves **high detection capability** with a strong balance between precision and recall. A high recall value indicates that most attacks were successfully detected, which is critical in cybersecurity systems. The relatively low false positive rate suggests that the system does not frequently misclassify normal traffic as malicious, reducing unnecessary alerts.

Compared to traditional machine learning approaches, the proposed method demonstrates improved generalization on unseen data, indicating better adaptability to real-world network environments.

5.4 Comparison in evaluation metrics

The quantitative evaluation of the proposed intrusion detection models is summarized in Table Y, which presents a comparative analysis of three experimental configurations: the NON-TL model (CNN trained from scratch on UNSW-NB15), the TL model (transfer learning from BoT-IoT to UNSW-NB15), and the base paper results reported by Rodríguez et al. (2022). This multi-faceted comparison serves to validate the effectiveness of the transfer learning approach while establishing the reproducibility and extension of prior work within the context of this research.

Table 14. Comparison of NON-TL, TL and BASE PAPER [11].

Metrics	NON-TL	TL	BASE PAPER
Accuracy	98.83%	99.09%	99.04%
Precision	98.70%	99.86%	99.06%
Recall	98.52%	98.53%	99.04%
F1- Score	98.61%	99.21%	99.05%

The experimental results demonstrate that the proposed transfer learning model achieves superior overall performance across all evaluation metrics when compared to the NON-TL baseline, while maintaining competitive results against the base paper. The TL model attains an accuracy of 99.09%, representing a 0.26% improvement over NON-TL (98.83%) and a marginal 0.05% enhancement over the base paper (99.04%). This accuracy improvement, though numerically modest, translates to approximately 260 additional correctly classified instances per 100,000 samples, a practically significant reduction in misclassifications for high-stakes security operations where every undetected attack carries potential consequences.

Most notably, the TL model achieves exceptional precision of 99.86%, substantially outperforming both NON-TL (98.70%) and the base paper (99.06%). This 1.16% improvement over NON-TL and 0.80% improvement over the base paper represents a critical advancement for practical deployment in Security Operations Center (SOC) environments. High precision indicates that when the model raises an alert, it is correct 99.86% of the time, meaning only 14 false alarms per 10,000 predictions. This dramatic reduction in false positives directly addresses the problem of analyst fatigue and alert desensitization identified in the problem statement. In operational contexts, false positives consume valuable analyst time and cognitive resources, potentially

leading to genuine threats being overlooked amidst excessive noise. The TL model's near-perfect precision ensures that security teams can trust and act upon alerts with high confidence, streamlining incident response workflows and maintaining situational awareness without overwhelming monitoring staff.

Recall, or detection rate, measures the model's ability to identify actual attacks, a metric of paramount importance given that missed detections represent security breaches. The TL model achieves 98.53% recall, marginally surpassing NON-TL (98.52%) by 0.01%, though falling 0.51% below the base paper's 99.04%. This slight reduction relative to the base paper warrants careful interpretation. The recall difference may be attributed to the subset sampling of the BoT-IoT dataset (approximately 25% of the full dataset) used for source domain training due to computational constraints. Reduced training data in the source domain may limit the model's exposure to the full diversity of attack patterns, consequently affecting recall on the target domain. Nevertheless, 98.53% recall remains highly acceptable for operational deployment, detecting 9,853 attacks out of every 10,000 at a rate that would identify the vast majority of security incidents while maintaining exceptional precision.

The F1-score, as the harmonic mean of precision and recall, provides a balanced measure of overall classification performance. The TL model achieves 99.21% F1-score, outperforming NON-TL (98.61%) by 0.60% and the base paper (99.05%) by 0.16%. This comprehensive metric confirms that transfer learning successfully optimizes the precision-recall trade-off, achieving better balance than both alternative configurations. The superior F1-score validates that the knowledge transferred from BoT-IoT enables the model to learn more robust decision boundaries that simultaneously minimize false positives (maintaining high precision) while maximizing true positives (maintaining competitive recall).

The comparison with NON-TL is particularly instructive for isolating the specific benefits of transfer learning. Both models share identical CNN architecture and are evaluated on the same UNSW-NB15 test set; the only difference lies in initialization random weights for NON-TL versus pretrained weights from BoT-IoT for TL. The consistent performance advantage of TL across all metrics conclusively demonstrates that transferred knowledge provides measurable benefits beyond what can be achieved through architectural design alone. The TL model achieves higher

accuracy (+0.26%), substantially higher precision (+1.16%), equivalent recall (+0.01%), and higher F1-score (+0.60%) improvements directly attributable to the effective reuse of features learned from the source domain.

The comparison with the base paper serves two purposes: validation of successful replication and demonstration of extension. The TL model marginally exceeds base paper accuracy (99.09% vs 99.04%) and substantially exceeds precision (99.86% vs 99.06%), while slightly underperforming in recall (98.53% vs 99.04%). These results confirm that the core transfer learning methodology has been successfully reproduced, while the enhanced precision indicates that this project's preprocessing pipeline, feature engineering, or fine-tuning strategy may have optimized the model differently than the original implementation. The higher precision at the cost of slightly reduced recall represents a design choice favoring reduced false alarms, a trade-off often preferred in operational SOC environments where analyst efficiency is prioritized.

From an operational perspective, the TL model's performance profile translates to tangible benefits for cybersecurity monitoring. The 99.86% precision means that security analysts can investigate alerts with confidence that nearly all will represent genuine threats, eliminating the "cry wolf" effect that plagues many intrusion detection systems. The 98.53% recall ensures that the vast majority of attacks trigger alerts, maintaining security posture without overwhelming monitoring staff. The combination of these metrics suggests that the proposed transfer learning framework achieves an optimal balance for real-world deployment, where both false positives and false negatives carry significant but different costs.

Chapter 6: Conclusion and Future Works

6.1 Conclusion

This project successfully presented the design, implementation, and evaluation of an AI-based Intrusion Detection System (IDS) for enhancing network security monitoring in IoT environments. The primary objective was to build a system capable of accurately distinguishing between normal and malicious network traffic using transfer learning techniques, while addressing the limitations of traditional visual-only alert mechanisms through the integration of audio feedback.

Throughout this project, several important stages were completed. First, relevant benchmark datasets—BoT-IoT and UNSW-NB15—were selected to represent realistic network traffic and modern cyberattacks. The BoT-IoT dataset served as the source domain containing diverse IoT-specific attack patterns, while UNSW-NB15 functioned as the target domain to evaluate the model's generalization capability across different network environments. Extensive data preprocessing was performed, including data cleaning to remove inconsistencies and missing values, feature selection to retain only the most relevant attributes, one-hot encoding for categorical variables, and normalization to ensure consistent feature scales suitable for deep learning.

A baseline Convolutional Neural Network (CNN-B) was developed and trained on the BoT-IoT source dataset, achieving strong performance with 99.98% test accuracy and minimal loss of 0.000399. The learned weights from this source model were then transferred to a target model (CNN-TL), which was fine-tuned on the UNSW-NB15 dataset using a lower learning rate and reduced training epochs to adapt to the target domain while preserving general features learned from the source.

The experimental results demonstrated that the proposed transfer learning system achieved superior performance compared to training from scratch. The CNN-TL model attained 99.09% accuracy, 99.86% precision, 98.53% recall, and a 99.21% F1-score, outperforming the non-transfer learning baseline (98.83% accuracy, 98.70% precision, 98.52% recall, 98.61% F1-score).

The substantial improvement in precision—1.16% higher than NON-TL—is particularly significant for operational deployment, as it dramatically reduces false alarms that contribute to analyst fatigue in Security Operations Center (SOC) environments. Per-class analysis revealed that the TL model achieved detection rates exceeding 99% for most attack categories, including analysis (100%), backdoor (99.14%), fuzzers (99.89%), and shellcode (99.82%), with only worms showing slightly lower detection at 96.88%.

Beyond detection performance, this project addressed the critical limitation of visual-only monitoring identified in the problem statement. A multi-modal alert system was implemented, integrating a real-time monitoring dashboard displaying detection results with confidence scores, a sonification module that translates detected intrusions into distinct audio cues based on attack type, and a Telegram bot notification system delivering immediate mobile alerts to designated recipients. This combination of visual, auditory, and mobile notifications ensures that security events can be perceived immediately through multiple channels, reducing response delays and enhancing situational awareness even when analysts are not actively monitoring visual displays.

The system's successful demonstration in a simulated environment confirmed its practical viability, with real-time traffic classification and immediate alert generation validating the integration of detection and notification components. This confirms that the proposed framework can effectively support SOC analysts by automatically flagging suspicious traffic, reducing manual workload, and enabling faster incident response.

In conclusion, this project proves that transfer learning techniques can significantly enhance traditional intrusion detection approaches, achieving superior accuracy with reduced training time and computational cost. The integration of audio feedback addresses a genuine operational need in cybersecurity monitoring, demonstrating that effective security solutions must consider not only detection performance but also how alerts are communicated to human operators. The research objectives—implementing transfer learning for improved attack detection, evaluating the model using standard IDS metrics, and integrating notification mechanisms to enhance alertness—have all been successfully achieved.

6.2 Limitations of the Study

Despite the promising performance and contributions of the proposed system, several limitations should be acknowledged when interpreting the results and considering future work.

1. Dataset Subset Utilization

Due to computational and resource constraints, only a subset of the BoT-IoT dataset—approximately 1 million records out of the full 72 million—was used for source domain training. While this subset was carefully selected to include major traffic and attack categories, it may not fully capture the complete diversity and complexity of real-world IoT network traffic. The reduced dataset size could potentially limit the model's exposure to rare attack patterns or edge cases, affecting its ability to generalize to highly imbalanced real-world scenarios. The slightly lower recall (98.53% vs. the base paper's 99.04%) may be partially attributable to this limitation.

2. Offline Experimental Environment

The model was developed, trained, and evaluated in an offline experimental environment using pre-collected datasets, rather than being deployed in a live network with streaming traffic. Factors critical to real-time operation—such as network latency, packet capture performance, throughput scalability under high traffic loads, and system responsiveness to streaming data—were not evaluated. These operational factors could significantly affect detection performance and response time when deployed in production environments.

3. Limited Real-Time Alert Evaluation

While the sonification and Telegram notification modules were successfully implemented and demonstrated, delays in the notification system following detection prevented a thorough evaluation of real-time alerting effectiveness. Quantitative metrics such as end-to-end latency from packet arrival to alert generation, audio response time, and notification delivery time were not systematically measured. This limits conclusions regarding the system's operational responsiveness for time-critical security incidents.

4. Absence of Automated Response Mechanisms

The current system focuses on detection and alerting but does not extend to automated incident response. In operational SOC environments, effective security monitoring requires not only accurate detection but also the ability to automatically correlate alerts with other security events, prioritize threats based on asset criticality, and trigger appropriate mitigation actions. The absence of integration with Security Information and Event Management (SIEM) platforms or orchestration tools limits the system's practical utility in full-scale security operations.

5. Zero-Day Attack Detection Capability

As a supervised learning approach, the proposed model's detection capability is inherently limited to attack types represented in the training data. While transfer learning improves generalization across domains, it does not fundamentally address the challenge of detecting zero-day attacks—previously unseen threats that exploit unknown vulnerabilities. The model's 96.88% detection rate for worms, the lowest among all attack categories, suggests that novel or highly variant attack patterns may pose challenges that supervised learning alone cannot fully address.

6. Model Interpretability

The CNN architecture employed in this study, while effective for feature learning and classification, offers limited interpretability regarding its decision-making process. Security analysts often require understanding of why specific traffic was classified as malicious to make informed response decisions and to continuously improve detection rules. The "black box" nature of deep learning models can hinder trust and adoption in security-critical applications where explainability is valued.

7. Computational Requirements

Although transfer learning reduces training time compared to training from scratch, the CNN-TL model still requires significant computational resources for inference, particularly when deployed at scale in high-throughput network environments. The model's suitability for resource-constrained edge devices or environments with limited processing capacity was not evaluated.

8. Sonification User Study

While the sonification module was implemented based on established design principles from the literature, formal user studies with SOC analysts were not conducted to evaluate the effectiveness of different audio mappings or to measure improvements in situational awareness and response time. The subjective and cognitive aspects of audio alert perception remain unexplored within the specific context of this system.

6.3 Future Work

The findings and contributions of this research open several promising avenues for future investigation and system enhancement. Building upon the successful implementation of the transfer learning-based intrusion detection framework with audio feedback, the following directions are proposed to extend the scope, improve performance, and facilitate real-world deployment of the system.

1. Real-Time Deployment and Operational Validation

While the proposed system has been thoroughly evaluated in an offline experimental environment using benchmark datasets, future work should focus on deploying the IDS within a live network infrastructure or operational Security Operations Center (SOC) environment. Such real-time deployment would enable comprehensive evaluation of critical operational parameters including detection latency, throughput scalability under high network traffic loads, and system responsiveness when processing streaming data. Performance metrics such as end-to-end processing time from packet capture to alert generation, maximum sustainable traffic volume without packet loss, and multi-threaded or distributed processing capabilities would provide valuable insights for production deployment. Additionally, deploying the system in a live SOC would allow assessment of how the audio feedback mechanism performs in actual monitoring environments, including user acceptance studies with security analysts evaluating the effectiveness of sonification for maintaining situational awareness during multitasking. Integration with existing network infrastructure components such as firewalls, switches, and traffic mirrors would also need to be validated to ensure seamless deployment without disrupting production operations.

2. Expanded Dataset Utilization and Diversity

The current study utilized approximately 25% of the BoT-IoT dataset for source domain training due to computational constraints. Future work should leverage the complete BoT-IoT dataset, comprising approximately 72 million records across 74 CSV files, to provide the transfer learning model with exposure to the full diversity of IoT attack patterns, including keylogging, data exfiltration, operating system and service scanning, and protocol-specific DDoS and DoS variations. Beyond expanding individual datasets, incorporating multiple benchmark datasets for both source and target domains would enhance model generalization and robustness. Promising candidates include CIC-IDS2017, CIC-IDS2019, CSE-CIC-IDS2018, and MQTT-IoT-IDS2020 datasets, each offering unique attack scenarios, traffic characteristics, and network environments. Multi-source transfer learning, where knowledge is distilled from multiple source domains simultaneously, could further improve adaptation to target domains by capturing broader feature representations. Additionally, cross-dataset validation—training on multiple datasets and testing on entirely unseen datasets—would provide rigorous assessment of model generalization to novel network environments.

3. Advanced Deep Learning Architectures and Hybrid Models

While the CNN-based architecture employed in this study demonstrated strong performance, future research should explore more sophisticated deep learning architectures to capture complex traffic behaviors and temporal dependencies. Recurrent Neural Networks (RNNs), particularly Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks, are well-suited for modeling sequential dependencies in network flows, potentially improving detection of multi-stage attacks that unfold over extended time periods. Convolutional LSTM (ConvLSTM) architectures could combine spatial feature extraction with temporal sequence modeling, capturing both local pattern correlations and temporal dynamics. Transformer-based architectures, which have revolutionized natural language processing and are increasingly applied to time-series analysis, offer powerful self-attention mechanisms for identifying long-range dependencies in network traffic sequences. Models such as the Temporal Fusion Transformer (TFT) or Informer could be adapted for intrusion detection, potentially capturing complex attack signatures spanning multiple packets or sessions. Hybrid architectures combining CNNs for spatial feature extraction with Transformers for temporal relationship modeling represent a particularly promising direction. Attention mechanisms, such as the Convolutional Block Attention Module (CBAM) employed in

related work by Abdelhamid et al. (2024), could be integrated to help models focus on the most discriminative features, potentially improving both accuracy and interpretability. Graph Neural Networks (GNNs) offer another intriguing direction by modeling network traffic as communication graphs, capturing relationships between communicating hosts that may reveal distributed attack patterns such as botnet command-and-control communication.

4. Zero-Day Attack Detection Through Unsupervised and Semi-Supervised Learning

A fundamental limitation of supervised learning approaches, including the transfer learning method presented in this study, is their dependence on labeled attack data for training. This constraint inherently limits detection capability to attack types represented in the training dataset, leaving systems vulnerable to zero-day exploits—previously unseen attacks that exploit unknown vulnerabilities. Future work should integrate unsupervised and semi-supervised learning techniques to enhance detection of novel threats. Autoencoder-based anomaly detection represents a promising approach, where models learn to reconstruct normal traffic patterns and flag significant reconstruction errors as potential anomalies indicative of unknown attacks. Variational Autoencoders (VAEs) and Denoising Autoencoders could further improve robustness by learning probabilistic representations of normal behavior. One-class classification methods, including One-Class SVM and Isolation Forests, offer alternative approaches for defining normal traffic envelopes and identifying outliers. Generative Adversarial Networks (GANs) could be employed to generate synthetic attack variations, augmenting training data and potentially improving model robustness to attack variants. More sophisticated approaches such as Deep Support Vector Data Description (Deep SVDD) or self-supervised learning techniques that learn representations by solving pretext tasks (e.g., predicting masked traffic features, contrastive learning of normal vs. anomalous patterns) could further enhance zero-day detection capability. Hybrid systems combining supervised transfer learning for known attack detection with unsupervised anomaly detection for unknown threat identification would provide comprehensive coverage against both familiar and novel attack vectors, significantly improving practical security posture.

5. Comprehensive SOC Integration and Security Orchestration

To maximize practical utility in real-world cybersecurity operations, the proposed system should be enhanced to integrate seamlessly with existing SOC infrastructure and workflows. Integration

with Security Information and Event Management (SIEM) platforms such as Splunk, IBM QRadar, or Elastic Stack would enable correlation of IDS alerts with logs from diverse sources including firewalls, endpoint detection systems, authentication servers, and application logs. Such correlation is essential for reducing false positives through contextual validation, prioritizing alerts based on asset criticality and threat intelligence, and constructing complete attack narratives for incident investigation. Automated alert enrichment using threat intelligence feeds (e.g., VirusTotal, AlienVault OTX, MISP) could provide real-time context about observed indicators of compromise, accelerating triage and response decisions. The system should support standardized alert formats such as Intrusion Detection Message Exchange Format (IDMEF) or Common Event Format (CEF) to ensure interoperability with diverse security tools.

Beyond passive alerting, future work should explore automated incident response capabilities. Integration with orchestration platforms such as Security Orchestration, Automation, and Response (SOAR) tools (e.g., Demisto, Phantom, Swimlane) could enable predefined response playbooks triggered by specific alert types. For example, upon detecting a brute-force attack, the system could automatically block the offending IP address at the firewall, terminate suspicious sessions, and initiate credential reset procedures for affected accounts. For DDoS attacks, automated traffic rerouting through scrubbers or cloud-based mitigation services could be triggered. Such automation would dramatically reduce mean time to respond (MTTR) and alleviate analyst workload during high-severity incidents. However, automated response must be implemented with careful safeguards, including human approval workflows for high-impact actions, rollback capabilities, and comprehensive audit logging to maintain accountability and prevent unintended consequences.

The audio feedback mechanism introduced in this study could be extended beyond simple alerting to provide sonification of security posture and threat intelligence. For instance, different audio patterns could indicate changes in overall risk scoring, ongoing attack campaigns relevant to the organization's industry sector, or unusual traffic patterns that have not yet crossed alert thresholds but warrant attention. Such continuous situational awareness sonification could help SOC analysts maintain peripheral awareness of network health while focusing on detailed investigations, addressing the multitasking needs identified in the sonification literature reviewed in Chapter 2.

6. Model Interpretability and Explainable AI

Deep learning models, including the CNN architecture employed in this study, are often criticized as "black boxes" whose decision-making processes are opaque to human analysts. This lack of interpretability poses challenges for security operations, where analysts must understand why specific alerts were generated to make informed response decisions and to continuously improve detection rules. Future work should integrate Explainable AI (XAI) techniques to provide insight into model predictions. Methods such as SHAP (SHapley Additive exPlanations), LIME (Local Interpretable Model-agnostic Explanations), and Grad-CAM (Gradient-weighted Class Activation Mapping) could be adapted to highlight which features most strongly influenced each classification decision. For example, when the model flags traffic as a DDoS attack, visualization techniques could indicate whether the decision was driven by packet rate, byte volume, source IP distribution, or protocol anomalies—helping analysts understand attack characteristics and appropriate response strategies. Attention mechanisms integrated into model architectures, as explored by Abdelhamid et al. (2024), not only improve performance but also provide interpretability by revealing which parts of the input the model focuses on. Such interpretability would enhance analyst trust in the system, facilitate model debugging and improvement, and support regulatory compliance requirements for explainable automated decisions in security-critical applications.

7. Federated Learning for Privacy-Preserving Collaborative Detection

The transfer learning approach in this study assumes centralized training on collected datasets, which may be impractical in scenarios where data privacy, regulatory constraints, or competitive concerns prevent organizations from sharing network traffic data. Federated Learning (FL) offers a compelling alternative, enabling multiple organizations to collaboratively train intrusion detection models without sharing raw data. Extending the current framework to a federated transfer learning approach, as explored by Fan et al. (2020), would allow each participating organization to train locally on its own data while periodically sharing model updates (gradients or weights) with a central coordinator that aggregates improvements into a global model. This approach would enable knowledge sharing across organizational boundaries while preserving data privacy and complying with regulations such as GDPR or HIPAA. Challenges to address include handling non-

IID (non-independent and identically distributed) data distributions across organizations, secure aggregation protocols to prevent gradient leakage, and communication efficiency for large-scale deployments. Successful implementation would enable intrusion detection systems to benefit from diverse attack experiences across multiple organizations without compromising sensitive network data.

8. Lightweight Edge Deployment for Resource-Constrained IoT Devices

While the current system is designed for SOC environments with substantial computational resources, many IoT deployments require intrusion detection capabilities at the network edge, where devices have severe constraints on processing power, memory, and energy consumption. Future work should explore model compression and optimization techniques to enable edge deployment of transfer learning-based detection. Approaches such as knowledge distillation (training smaller "student" models to mimic larger "teacher" models), quantization (reducing numerical precision of weights), pruning (removing unimportant connections), and neural architecture search (automatically designing efficient architectures) could produce lightweight models suitable for resource-constrained environments. Such edge-deployed IDS could provide real-time detection at the source, reducing latency, conserving network bandwidth, and maintaining protection even when connectivity to central SOC is interrupted. The audio feedback mechanism could be adapted for local alerting on edge devices, providing immediate notification to nearby personnel or integration with local alarm systems.

9. Adversarial Robustness Evaluation and Enhancement

As machine learning-based IDS become more prevalent, they increasingly become targets for adversarial attacks designed to evade detection or cause false alarms. Future work should systematically evaluate the proposed system's robustness against adversarial examples—carefully crafted perturbations to network traffic designed to fool the classifier while remaining functionally malicious. Techniques such as Fast Gradient Sign Method (FGSM), Projected Gradient Descent (PGD), and Carlini-Wagner attacks could be adapted to the network traffic domain to assess vulnerability. Adversarial training, where the model is trained on both clean and adversarially perturbed examples, could enhance robustness. Detection of adversarial attempts through statistical analysis of traffic features or ensemble methods could provide additional defense layers.

Understanding and improving adversarial robustness is essential for deployment in high-stakes environments where sophisticated attackers may actively attempt to subvert detection systems.

10. Longitudinal Evaluation and Continuous Learning

Network traffic patterns evolve over time as new applications emerge, user behaviors change, and attack methodologies advance. The static model training employed in this study, while appropriate for controlled evaluation, does not account for temporal drift in data distributions. Future work should implement continuous learning mechanisms that periodically retrain or fine-tune models on newly collected data to maintain detection performance over extended deployment periods. Techniques such as online learning, where models update incrementally as new data arrives, or periodic retraining with adaptive window sizes could be explored. Longitudinal studies tracking model performance over months or years would provide valuable insights into model decay rates, optimal retraining frequencies, and the evolving nature of network traffic in real-world environments.

6.4 Project Reflection

Undertaking this final year project has been an invaluable learning experience that has significantly enriched my understanding of cybersecurity, machine learning, and the research process. As a bachelor's student, this project represents the culmination of my undergraduate studies, integrating knowledge acquired across multiple disciplines while exposing me to the challenges and rewards of conducting independent research.

Technical Knowledge Acquisition

The most immediate and tangible learning outcome has been the deep technical knowledge acquired in areas previously encountered only superficially in coursework. Prior to this research, my understanding of intrusion detection systems was limited to theoretical concepts—I could explain the difference between signature-based and anomaly-based detection but had never implemented or evaluated such systems. This project required me to move from passive learning to active application, designing and building a functioning IDS capable of classifying network traffic with high accuracy. Through this process, I gained practical proficiency in machine learning frameworks such as TensorFlow and Keras, learning to construct, train, and evaluate convolutional

neural networks while troubleshooting issues ranging from tensor shape mismatches to vanishing gradients.

The transfer learning component introduced me to sophisticated concepts in deep learning that extend beyond typical undergraduate curriculum. Understanding how to freeze layers, fine-tune pretrained models, and adapt knowledge across domains required me to engage with primary research literature, including the base paper by Rodríguez et al. (2022) and related works. Reading and comprehending academic papers was itself a learned skill—initially challenging, but progressively more manageable as I developed the ability to extract key methodologies, identify contributions, and critically evaluate experimental designs.

The dataset preprocessing phase, while less glamorous than model design, proved to be one of the most educational aspects of the project. Working with the BoT-IoT and UNSW-NB15 datasets exposed me to the realities of real-world data: missing values, inconsistent formatting, categorical variables requiring encoding, and skewed distributions demanding transformation. This taught me that successful machine learning depends as much on data quality as on algorithmic sophistication.

Research Methodology and Experimental Design

This project provided my first experience with formal research methodology, teaching me the importance of systematic experimental design for producing reliable, reproducible results. I learned to formulate clear research objectives aligned with identified problems, design experiments that isolate specific variables, and select appropriate evaluation metrics that comprehensively assess performance. The comparative analysis between NON-TL and TL models exemplifies this methodological rigor—by controlling all factors except weight initialization, I could attribute performance differences specifically to transfer learning rather than architectural variations.

The process of documenting my methodology taught me the value of reproducibility in research. Writing the methodology chapter required me to describe each step with sufficient detail that another researcher could replicate my work. This discipline of thorough documentation ensures that research contributions are verifiable and can serve as foundations for future work.

Problem-Solving and Debugging Skills

Perhaps the most practically valuable skills I developed relate to problem-solving and debugging. Implementing a complex system involving data pipelines, model training, evaluation metrics, and notification integration inevitably encountered numerous challenges. Tensor shape mismatches between convolutional layers and dense layers required careful calculation of flatten dimensions. Training instability necessitated learning rate adjustments and the addition of dropout layers. Memory constraints with large datasets forced creative sampling strategies and batch size optimization. Each obstacle required systematic debugging—formulating hypotheses, testing potential solutions, and iteratively refining approaches until successful.

One particularly instructive challenge involved ensuring feature compatibility between source and target domains for transfer learning. The BoT-IoT and UNSW-NB15 datasets, while both containing network traffic features, had different feature sets that required alignment through careful feature selection and the insertion of auxiliary columns to maintain consistent input dimensions. Solving this problem required me to deeply understand both datasets, the requirements of the CNN architecture, and the principles of transfer learning—integrating knowledge from multiple domains to devise a practical solution.

Understanding the Cybersecurity Landscape

Beyond technical skills, this project deepened my appreciation for the complexity and importance of cybersecurity. Researching the growth of IoT and emerging security risks, including the casino fish tank thermometer attack, illustrated how the expanding attack surface creates vulnerabilities in unexpected places. Understanding the pressures faced by SOC analysts—information overload, alert fatigue, the cognitive burden of continuous monitoring—gave me empathy for the human element in cybersecurity and motivated the audio feedback contribution of this project.

This project also taught me that cybersecurity is not purely a technical challenge but involves human factors, organizational processes, and economic considerations. A technically perfect intrusion detection system that generates too many false positives will be ignored or disabled by analysts. A system that requires constant visual attention cannot support the multitasking demands

of real SOC environments. These insights have shaped my understanding of what makes security solutions effective in practice—not just high accuracy metrics, but usability, integration with workflows, and support for human decision-making.

Time Management and Project Planning

As a significant independent project spanning multiple months, this research required careful time management and project planning. I learned to break a complex undertaking into manageable phases: literature review, methodology design, implementation, experimentation, analysis, and writing. Each phase had its own challenges and dependencies—implementation could not begin until methodology was defined, analysis could not proceed until experiments completed. Creating and adhering to a project timeline, while accommodating unexpected delays, taught me realistic project management skills applicable to any future professional work.

Academic Writing and Communication

Writing this thesis has significantly improved my academic writing and communication skills. Translating technical work into clear, structured prose that communicates methodology, results, and significance to readers requires different skills than writing code. I learned to structure arguments logically, support claims with evidence, and maintain consistent terminology throughout. Crafting figure and table explanations that guide readers through visual information taught me to anticipate reader questions and provide context that makes technical content accessible.

Personal Growth and Confidence

Finally, this project has contributed significantly to my personal growth and confidence as an emerging professional. Completing a substantial independent research project from conception to completion has demonstrated my capability to tackle complex, open-ended problems without predetermined solutions. The challenges overcome, skills acquired, and knowledge generated have convinced me that I can contribute meaningfully to the field.

The project has also clarified my interests and career direction. The intersection of cybersecurity and machine learning represents a dynamic field with tremendous growth potential and societal

importance. The experience of building an intrusion detection system, understanding attacker methodologies, and contributing to defensive capabilities has confirmed my desire to pursue a career in cybersecurity.

6.5 Final Remarks

In reflection, this final year project has been far more than an academic requirement—it has been a transformative learning experience that integrated technical knowledge, research methodology, problem-solving skills, and personal development. The technical competencies gained in machine learning, deep learning, and intrusion detection complement my undergraduate coursework and provide a foundation for future learning. The research skills developed—literature review, experimental design, critical analysis, academic writing—prepare me for graduate study or research-oriented industry roles. The personal growth in confidence, perseverance, and professional identity positions me to contribute meaningfully to the cybersecurity field.

As stated in the conclusion of this thesis, AI-driven intrusion detection systems have strong potential to become core components of future intelligent SOC environments. Through this project, I have contributed to that vision while simultaneously developing the knowledge, skills, and confidence to participate in realizing it. The challenges overcome and lessons learned will serve me well throughout my career, whatever direction it may take. I am grateful for the opportunity to undertake this research and for the guidance of my supervisor, Madam Y.M. Raja Azlina Binti Raja Mahmood, whose support made this learning journey possible.

References

- [1] L. Axon, J. Happa, M. Goldsmith, and S. Creese, "Hearing attacks in network data: An effectiveness study," *Computers & Security*, vol. 83, pp. 367–388, Jun. 2019, doi: 10.1016/j.cose.2019.03.004.
- [2] Cristian, "Transfer Learning," Medium, Sep. 24, 2020. [Online]. Available: https://medium.com/@1154_75881/transfer-learning-628e83df5c8a
- [3] M. A. Rahman and M. S. Sadi, "IoT Enabled Automated Object Recognition for the Visually Impaired," *Computer Methods and Programs in Biomedicine Update*, vol. 1, p. 100015, 2021, doi: 10.1016/j.cmpbup.2021.100015.
- [4] C. Sabino, L. A. Silva, A. Fernandes, G. V. González, V. R. Q. Leithardt, and W. D. Parreira, "Intelligent sensors in assistive systems for deaf people: a comprehensive review," *PeerJ Computer Science*, vol. 10, p. e2411, 2024, doi: 10.7717/peerj-cs.2411.
- [5] B. Semeniuk, A. Kashtalian, D. Martiniuk, A. Drozd, and A.-B. Salem, "Detection of computer attacks based on sonification of network traffic," [Journal Name Missing], 2025. [Manuscript submitted for publication / Conference proceeding - please verify].
- [6] S. Sinha, "State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally," *IoT Analytics*, Sep. 3, 2024. [Online]. Available: <https://iot-analytics.com/number-connected-iot-devices/>

- [7] M. Vargas, B. Kapralos, M. Green, M. Garcia-Ruiz, and A. Brown, "Towards Music-Assisted Intrusion Detection," in [Conference Name if known] , 2009. [Poster presentation - please verify source].
- [8] W. Wei, "Casino Gets Hacked Through Its Internet-Connected Fish Tank Thermometer," The Hacker News, Apr. 16, 2018. [Online]. Available: <https://thehackernews.com/2018/04/iot-hacking-thermometer.html>
- [9] P. Williams, I. K. Dutta, H. Daoud, and M. Bayoumi, "A Survey on Security in Internet of Things with a Focus on the Impact of Emerging Technologies," Internet of Things, vol. 19, p. 100564, Aug. 2022, doi: 10.1016/j.iot.2022.100564.
- [10] World Health Organization, "World report on vision," WHO, Geneva, 2019. [Online]. Available: <https://www.who.int/publications/i/item/world-report-on-vision>
- [11] I. A. Fares, A. G. A. Ibrahim, M. A. Elaziz, M. Shrahili, A. A. Elmahallawy, R. M. Sohaib, M. A. Shawky, and S. T. Shah, "Deep Transfer Learning Based on Hybrid Swin Transformers With LSTM for Intrusion Detection Systems in IoT Environment," IEEE Open Journal of the Communications Society, vol. 6, pp. 4342-4365, 2025, doi: 10.1109/OJCOMS.2025.3569301.
- [12] J. Mwaura, S. Araki, M. B. Musthafa, S. Huda, and Y. Nogami, "A Study on Transfer Learning TinyML-Based Intrusion Detection Framework on IoT Devices," in 2025 IEEE International Conference on Consumer Electronics (ICCE), Las Vegas, NV, USA, 2025, pp. 1-6, doi: 10.1109/ICCE63647.2025.10929917.

- [13] A. Alfahaid, E. Alalwany, A. M. Almars, F. Alharbi, E. Atlam, and I. Mahgoub, "Machine Learning-Based Security Solutions for IoT Networks: A Comprehensive Survey," *Sensors*, vol. 25, no. 11, p. 3341, 2025, doi: 10.3390/s25113341.
- [14] A. Salehiyan, P. S. Moghaddam, and M. Kaveh, "An Optimized Transformer–GAN–AE for Intrusion Detection in Edge and IIoT Systems: Experimental Insights from WUSTL-IIoT-2021, EdgeIIoTset, and TON_IoT Datasets," *Future Internet*, vol. 17, no. 7, p. 279, 2025, doi: 10.3390/fi17070279.
- [15] L. Raja, G. Sakthi, S. Vimalnath, and G. Subramaniam, "An improved federated transfer learning model for intrusion detection in edge computing empowered wireless sensor networks," *Concurrency and Computation: Practice and Experience*, 2024, doi: 10.1002/cpe.8259.
- [16] E. Baraka (Supervisor), "Framework for Next Generation Security Operation Center Powered by Artificial Intelligence," Ph.D. Dissertation, College of Information Technology, UAE University, 2025. [Online]. Available: https://scholarworks.uaeu.ac.ae/thesis_dissertation_defense/2025-2024/dissertation_defenses/59/
- [17] Y. Lee, J. Im, J. Kim, and M. Yoon, "Mitigating semantic label divergence in federated learning: Obfuscated encoding and alert filtering for security monitoring," *PLoS One*, vol. 20, no. 12, p. e0338488, Dec. 2025, doi: 10.1371/journal.pone.0338488.
- [18] Y. Feng and K. Sakurai, "Network Intrusion Detection: Evolution from Conventional Approaches to LLM Collaboration and Emerging Risks," 2025. [Online]. Available: arXiv:2510.23313v2.
- [19] K. T. W. Teuwen, T. Mulders, E. Zambon, and L. Allodi, "Ruling the Unruly: Designing Effective, Low-Noise Network Intrusion Detection Rules for Security Operations Centers," in *Proc. ACM Asia Conference on Computer and Communications Security (ASIA CCS '25)*, Hanoi, Vietnam, 2025, doi: 10.1145/3708821.3710823.

- [20] N. Daniel, F. K. Kaiser, S. Giladi, S. Sharabi, R. Moyal, S. Shpolyansky, A. Murillo, A. Elyashar, and R. Puzis, "Labeling Network Intrusion Detection System (NIDS) Rules with MITRE ATT&CK Techniques: Machine Learning vs. Large Language Models," *Big Data and Cognitive Computing*, vol. 9, no. 2, p. 23, 2025, doi: 10.3390/bdcc9020023.
- [21] "AuraCast: A Multisensory Network Monitoring System for Cybersecurity - Integrating Sonification and IoT-Enhanced Visualisation to Combat Alert Fatigue," Master's Thesis, University of Agder, 2025. [Online]. Available: <https://uia.brage.unit.no/uia-xmlui/handle/11250/3209122>
- [22] D. Too, L. Axon, I. Agrafiotis, M. Goldsmith, and S. Creese, "Visualisation for cyber situational awareness in maritime: A field study of practitioner requirements," *Computers & Security*, vol. 154, p. 104433, 2025, doi: 10.1016/j.cose.2025.104433.
- [23] M. Sarhan, S. Layeghy, and M. Portmann, "NetFlow Datasets for Machine Learning-based Network Intrusion Detection Systems," in *Proc. BDTA 2020*, 2020. [Online]. Available: arXiv:2011.09144.
- [24] G. F. Monkam, M. J. De Lucia, and N. D. Bastian, "Detecting data poisoning attacks using topological data analysis and unsupervised learning for network intrusion detection," *Computers and Security*, vol. 144, p. 103958, Sep. 2024, doi: 10.1016/j.cose.2024.103958.
- [25] L. Axon, J. Saunders, P. Esteve González, J. Carver, W. Dutton, M. Goldsmith, and S. Creese, "Cyber-defence partnerships: Ukraine as a test case of private-public collaboration," *Journal of Cyber Policy*, vol. 9, no. 3, pp. 399-422, 2024, doi: 10.1080/23738871.2024.2435452.

Appendix